



PROJECT REPORT

DESIGN & DEVELOPMENT OF AN INTELLIGENT AUTONOMOUS MOBILE ROBOT NAVIGATION SYSTEM USING THE A* ALGORITHM

— A **MATLAB**-BASED SIMULATION AND —
PERFORMANCE ANALYSIS STUDY —



COURSE MODULE
**Digital Simulation of
Intelligent Systems**



STUDENT NAME : Rathu Baduge Pamodya Kavindi



STUDENT ID : W2410816013



CHINESE NAME : 莉莎



MAJOR : Artificial Intelligence &
Automation Engineering



UNIVERSITY : Tiangong University



SUBMISSION DATE : 30 June 2026

START



GOAL

MATLAB SIMULATION OUTPUT



“ INTELLIGENT PATHS.
AUTONOMOUS FUTURES. ”

ACKNOWLEDGEMENT

In the first place, I wish to acknowledge and appreciate all those who have helped in making this project “Autonomous Mobile Robot Path Planning Using A* Algorithm with MATLAB GUI Simulation and Performance Analysis” * a success.

I am highly indebted to the lecturers and instructors of the Digital Simulation Intelligence System course, whose help made this project possible in terms of knowledge acquisition and support.

I must also extend my thanks to Tiangong University and the Department of Artificial Intelligence & Automation Engineering for the facilities they have given me which have made this project possible.

The contribution of all those academicians who have been directly or indirectly responsible for building up my knowledge in robotics, automation, simulation, and artificial intelligence cannot go unnoticed.

I would also like to thank my friends and classmates for the constructive suggestions they offered me during the development of this project.

Last but not the least, I am very grateful to my parents for the continued encouragement and motivation that they have given me throughout my academic career.

ABSTRACT

Autonomous mobile robots have become an important component in modern automation systems, where efficient and intelligent navigation is required in dynamic and structured environments. Path planning is one of the fundamental challenges in autonomous robotics, as the robot must determine an optimal path from a starting position to a target destination while avoiding obstacles.

This project presents the design and implementation of an Autonomous Mobile Robot Path Planning System using the A* (A-Star) Algorithm in a MATLAB simulation environment. A graphical user interface (GUI) was developed to allow users to create maps, select start and goal positions, execute path planning operations, and visualize robot navigation in real time.

The proposed system generates an obstacle-based environment and applies the A* algorithm to calculate the shortest feasible path between the selected locations. The generated path is displayed graphically, and robot movement is animated to simulate autonomous navigation. In addition, performance evaluation metrics such as path length and execution time are calculated and displayed to assess the efficiency of the algorithm.

To further enhance the analysis, graphical visualization techniques were incorporated through a path length analysis module and an algorithm comparison module. The performance of the A* algorithm was compared with the Dijkstra algorithm using execution time measurements, demonstrating the computational efficiency of the A* approach for robot path planning applications.

To extend the system's practical applicability, the project was further developed into a three-dimensional (3D) navigation framework. The environment was upgraded from a single layer 2D grid to a full volumetric occupancy grid, and the A* algorithm was extended to support 26-directional movement with Euclidean cost evaluation, enabling true diagonal travel through height-varying obstacle fields. A line-of-sight path smoothing technique was also implemented to eliminate redundant waypoints and generate more efficient routes. The system was further extended to support multi-robot coordination, in which three robots navigate the same 3D environment simultaneously using space-time reserved A* search strategy to plan collision-free paths in real time.

The results obtained from multiple simulation tests indicate that the developed system successfully generates optimal navigation paths in both single-robot and multi-robot scenarios, across two-dimensional and three-dimensional environments, while providing effective visualization and performance analysis capabilities. This project demonstrates the practical application of artificial intelligence, simulation techniques, and autonomous navigation principles in mobile robotics.

Keywords: Autonomous Mobile Robot, Path Planning, A* Algorithm, MATLAB GUI, Robot Navigation, Simulation, Artificial Intelligence, Dijkstra Algorithm, Performance Analysis, 3D Navigation, Multi-Robot Path Planning, Space-Time Search.

TABLE OF CONTENT

List of Tables.....	7
List of Figures.....	8
1: Introduction.....	9
1.1 Background of Autonomous Mobile Robots.....	9
1.2 Path Planning Concept.....	10
1.3 Importance of Robot Navigation.....	10
1.4 Problem Statement.....	11
1.5 Purpose of the Project.....	12
1.6 Objectives.....	12
1.7 Scope of the Project.....	13
1.8 Important of the Project.....	13
1.9 Structure of the report.....	14
2: Literature Review.....	15
2.1 Introduction.....	15
2.2 Autonomous Mobile Robots.....	15
2.3 Methods of Robot Path Planning	16
2.3.1 Classification of Path Planning.....	17
2.3.2 Important of Path Planning.....	17
2.4 Dijkstra Algorithm.....	18
2.5 A* Algorithm.....	18
2.6 Rapidly Exploring Random Tree (RRT).....	20
2.7 Genetic Algorithm-Based Path Planning.....	21
2.8 MATLAB for Robot Simulation.....	22
2.9 Comparison of Path Planning Algorithms.....	23
2.10 Summary of Literature Review.....	24

3: Methodology.....	25
3.1 System Overview.....	25
3.2 Project Development Process.....	25
3.3 Project Workflow.....	26
3.4 Environment Modeling.....	26
3.5 Obstacle Generation.....	27
3.6 Start and Goal Selection.....	27
3.7 A* Algorithm Implementation.....	28
3.7.1 Open List.....	29
3.7.2 Closed List.....	29
3.7.3 Cost Function.....	29
3.7.4Heuristic Function.....	29
3.8 Path Generation Process.....	30
3.9 Robot Animation Process.....	31
3.10 Performance Evaluation Method.....	32
3.11 Methodology Flowchart.....	33
4: System Design.....	35
4.1 System Architecture.....	35
4.2 GUI Design.....	36
4.3 Functional Modules.....	38
4.4 Data Flow Diagram (DFD).....	38
4.5 Use Case Diagram.....	39
5: Implementation.....	41
5.1 Software Requirements.....	41
5.2 MATLAB Development Environment.....	42
5.3 Project Structure.....	44
5.4 GUI Implementation.....	45
5.5 A* Algorithm Code Implementation.....	47

5.6 Robot animation Implementation.....	48.
5.7 Graph Visualization Implementation.....	50
5.8 Algorithms Comparison Implementation.....	51
6: Results and Analysis.....	53
6.1 Simulation Environment.....	53
6.2 Test Case 1.....	54
6.3 Test Case 2.....	55
6.4 Test Case 3.....	57
6.5 Path Length Analysis.....	58
6.6 Execution Time Analysis.....	59
6.7 Performance Graph Analysis.....	60
6.8 Discussion of Results.....	61
7: Conclusion and Future Work.....	62
7.1 Project Summary.....	62
7.2 Achievements.....	62
7.3 Limitations.....	63
7.4 Future Improvements.....	63
7.5 Conclusion.....	64
7.6 References	65
8: Extension to Three-Dimensional Multi-Robot Path Planning.....	66
8.1 Motivation for the Three-Dimensional Extension.....	66
8.2 Three-Dimensional Environment Modeling.....	66
8.3 Twenty-Six Directional A* Search.....	68
8.4 Line-of-Sight Path Smoothing.....	68
8.5 Multi-Robot Path Planning Using Space-Time Reservation.....	70
8.6 Synchronized Three-Dimensional GUI and Robot Animation.....	71
8.7 Results and Discussion.....	72
8.8 Chapter Summary.....	72

List of Tables

No.	Caption
Table 1.1	Structure of the Report
Table 2.1	Classification of Robot Path Planning Methods
Table 2.2	Advantages and Limitations of the Dijkstra Algorithm
Table 2.3	Advantages and Limitations of the A* Algorithm
Table 2.4	Advantages and Limitations of the RRT Algorithm
Table 2.5	Advantages and Limitations of the Genetic Algorithm
Table 2.6	Advantages of MATLAB for Robot Simulation
Table 2.7	Comparison of Path Planning Algorithms
Table 3.1	Project Development Process
Table 3.2	Evaluation Metric Summary
Table 4.1	GUI Components
Table 4.2	Functional Modules of The Proposed System
Table 5.1	Software Components
Table 5.2	Graphic Visualization Function
Table 5.3	Hardware Requirements
Table 5.4	Development Tool Summary
Table 5.5	MATLAB Development Features
Table 5.6	Project Module Structure
Table 5.7	Main GUI Components
Table 5.8	A* Algorithm Implementation Component
Table 5.9	Robot Animation Implementation Components
Table 5.10	Graph Visualization Components
Table 5.11	Algorithm Comparison Component
Table 6.1	Experimental Configuration
Table 6.2	Test Case 1 Configuration
Table 6.3	Test Case 2 Configuration
Table 6.4	Test Case 3 Configuration

Table 6.5	Path Length Results
Table 6.6	Execution Time Results
Table 6.7	Overall Performance Summary
Table 6.8	Summary of Experimental Findings
Table 7.1	Project Achievements
Table 7.2	Future Improvements
Table 8.1	Three-Dimensional Environment Modeling Parameters
Table 8.2	Comparison of Two-Dimensional and Three-Dimensional A* Search
Table 8.3	Line-of-Sight Path Smoothing Components
Table 8.4	Multi-Robot Space-Time A* Implementation Components

List of Figures

	Caption
Figure 5.1	MATLAB R2014b Development Environment
Figure 5.2	Main Graphical User Interface
Figure 5.3	Collision-Free Path Generate Using A* Algorithm
Figure 5.4	Robot Animation During Navigation
Figure 5.5	Path Length Analysis Graph
Figure 5.6	Comparison of A* and Dijkstra Algorithm
Figure 6.1	Simulation Environment
Figure 6.2	Generated Collision-Free Path for Test Case 1
Figure 6.3	Generated Collision-Free Path for Test Case 2
Figure 6.4	Generated Collision-Free Path for Test Case 3
Figure 6.5	Path Length Analysis Graph
Figure 6.6	Execution Time Results Displayed in the MATLAB GUI
Figure 8.1	Three-Dimensional Grid Environment with Randomly Generated Obstacles
Figure 8.2	Raw A* Path Compared with the Smoothed Line-of-Sight Path

Figure 8.3

Synchronized
Animation
Environment

Within

Multi-Robot
the
3-D

1: Introduction

1.1 Background of Autonomous Mobile Robots

Autonomous Mobile Robots (AMRs) are intelligent robotic systems capable of navigating and performing assigned tasks without continuous human intervention. Unlike conventional mobile robots, AMRs can perceive their surroundings, make navigation decisions, and move safely within an environment using path planning and obstacle avoidance techniques. These capabilities have made autonomous mobile robots an essential technology in modern industrial automation, healthcare, warehouse logistics, agriculture, and service robotics.

The effectiveness of an autonomous mobile robot depends largely on its navigation system. A reliable navigation system enables the robot to determine an optimal route from a starting location to a target destination while avoiding obstacles and minimizing travel distance and execution time. As robotic applications continue to expand, efficient navigation has become one of the most significant challenges in autonomous system development.

Recent advances in robotics, artificial intelligence, and computational technologies have significantly improved the capabilities of autonomous navigation systems. Various path planning algorithms have been developed to enhance navigation performance, among which the A* path planning algorithm is widely recognized for its efficiency, optimality, and suitability for grid-based environments. The algorithm combines actual travel cost and heuristic estimation to determine an optimal collision-free path while reducing unnecessary search operations.

In this project, an Intelligent Autonomous Mobile Robot Navigation System is developed using the MATLAB simulation environment. The proposed system integrates the A* path planning algorithm with a Graphical User Interface (GUI), robot animation, graph visualization, and performance evaluation modules to create an interactive platform for autonomous navigation. The system enables users to generate navigation environments, simulate robot movement, visualize the generated path, and evaluate navigation performance using path length and execution time.

The development of this project demonstrates the practical application of autonomous path planning techniques while providing a flexible simulation platform for studying intelligent robot navigation and evaluating path planning algorithms under different environmental conditions.

1.2 Path Planning Concept

Path planning is one of the fundamental functions of an autonomous mobile robot, enabling it to determine an efficient route from a starting position to a target destination while avoiding obstacles. The primary objective of path planning is to generate a collision-free path that minimizes travel distance, execution time, and computational cost while ensuring safe navigation within the operating environment.

Path planning techniques are generally classified into global and local approaches. Global path planning assumes that the environment is known in advance and calculates an optimal route before navigation begins. In contrast, local path planning allows the robot to respond dynamically to changes in the surrounding environment during navigation. The selection of an appropriate path planning technique depends on the application requirements and the characteristics of the operating environment.

Among the various path planning algorithms, the A* algorithm is widely used because of its efficiency and ability to generate optimal paths in grid-based environments. The algorithm combines the actual cost from the starting point with a heuristic estimate of the remaining distance to determine the most efficient route. This approach reduces unnecessary search operations while maintaining accurate path planning performance.

In this project, the A* path planning algorithm is implemented within the MATLAB simulation environment to generate collision-free navigation paths for an autonomous mobile robot. The generated path is visualized through the graphical user interface (GUI), enabling users to observe robot movement and evaluate navigation performance using path length and execution time measurements.

1.3 Importance of Robot Navigation

Robot navigation is a fundamental capability that enables autonomous mobile robots to move safely and efficiently within an environment while completing assigned tasks without continuous human intervention. An effective navigation system allows a robot to identify an optimal path, avoid obstacles, and reach its destination accurately, making it an essential component of intelligent robotic systems.

With the rapid advancement of automation technologies, robot navigation has become increasingly important in various application domains, including industrial manufacturing, warehouse automation, healthcare, agriculture, and service robotics. In these environments, autonomous navigation improves operational efficiency, reduces human effort, minimizes navigation errors, and enhances workplace safety. Reliable navigation systems also enable robots to operate in environments that may be hazardous or difficult for humans.

The performance of an autonomous mobile robot is directly influenced by the efficiency of its navigation algorithm. A well-designed path planning algorithm minimizes travel distance and execution time while ensuring collision-free movement. Therefore, selecting an appropriate navigation strategy is essential for improving overall system performance and reliability.

In this project, robot navigation is achieved through the implementation of the A* path planning algorithm within a MATLAB-based simulation environment. The developed system provides an interactive platform for generating optimal navigation paths, visualizing robot movement, and evaluating navigation performance. This demonstrates the practical importance of intelligent robot navigation and its potential for future real-world robotic applications.

1.4 Problem Statement

Autonomous mobile robots are increasingly used in applications such as industrial automation, warehouse management, healthcare, and service robotics, where safe and efficient navigation is essential. However, autonomous navigation remains a challenging task because robots must determine an optimal path while avoiding obstacles and

minimizing travel distance and execution time. Inefficient path planning may result in longer routes, increased computational cost, and reduced overall system performance.

Many conventional navigation methods either fail to generate optimal paths or require significant computational resources, particularly in complex environments. In addition, the absence of an interactive simulation platform limits the ability to visualize robot movement and evaluate navigation performance effectively. These limitations make it difficult to compare path planning algorithms and analyze their efficiency under different environmental conditions.

To address these challenges, this project proposes the development of an Intelligent Autonomous Mobile Robot Navigation System using the A* path planning algorithm within the MATLAB environment. The proposed system integrates path planning, robot animation, graph visualization, and performance evaluation into a single Graphical User Interface (GUI). This enables users to generate collision-free paths, observe autonomous robot movement, and evaluate navigation performance using execution time and path length measurements.

The proposed solution provides a practical simulation platform for studying autonomous robot navigation while demonstrating the effectiveness of the A* algorithm for efficient path planning in a grid-based environment.

1.5 Purpose of the Project

The purpose of this project is to design and develop an Intelligent Autonomous Mobile Robot Navigation System that demonstrates efficient and reliable path planning using the A* algorithm within the MATLAB simulation environment. The project aims to provide a user-friendly platform for simulating autonomous robot navigation, enabling users to create different navigation scenarios and evaluating robot performance under various environmental conditions.

The proposed system integrates path planning, obstacle avoidance, robot animation, graph visualization, and performance evaluation into a single Graphical User Interface (GUI). This integration allows users to generate optimal collision-free paths, visualize robot movement, and analyze navigation performance using key metrics such as execution time and path length.

In addition, the project serves as an educational and research platform for understanding autonomous navigation concepts and evaluating the effectiveness of the A* path planning algorithm. The developed system also provides a foundation for future enhancements, including dynamic obstacle avoidance, real-world robot implementation, sensor integration, and advanced path planning techniques.

1.6 Objectives

The primary objectives of this project are as follows:

1. To design and develop an Intelligent Autonomous Mobile Robot Navigation System using the MATLAB simulation environment.
2. To implement the A* path planning algorithm for generating optimal collision-free paths in a two-dimensional grid-based environment.
3. To develop an interactive Graphical User Interface (GUI) for creating simulation environments and controlling the navigation process.
4. To simulate autonomous robot movement through real-time animation and visualize the generated navigation path.
5. To evaluate the performance of the proposed system using execution time and path length as key performance metrics.
6. To compare the performance of the A* algorithm with the Dijkstra algorithm under identical simulation conditions.
7. To provide an interactive simulation platform that supports learning, research, and future development in autonomous mobile robot navigation.

1.7 Scope of the Project

The scope of this project focuses on the design, development, and evaluation of an Intelligent Autonomous Mobile Robot Navigation System using the MATLAB simulation environment. The proposed system implements the A* path planning algorithm to generate optimal collision-free paths in a two-dimensional grid-based environment containing static obstacles.

The project includes the development of an interactive Graphical User Interface (GUI) that allows users to create simulation environments, define start and goal positions, execute the path planning algorithm, visualize robot movement through real-time animation, and evaluate navigation performance using execution time and path length measurements. In addition, the system provides graphical visualization, and algorithm comparison features to analyze the performance of the implemented A* algorithm against the Dijkstra algorithm.

The scope of this project is limited to a simulation-based implementation and does not include dynamic obstacle avoidance, real-world robot deployment, sensor integration, three-dimensional navigation, or machine learning-based path planning techniques. These features are considered potential enhancements for future development.

1.8 Importance of the Project

The development of autonomous mobile robot navigation systems has become increasingly important due to the growing demand for intelligent automation across the various industries. Efficient navigation enables robots to perform tasks accurately, safely, and with minimal human intervention, making autonomous robots valuable in applications such as warehouse automation, manufacturing, healthcare, agriculture, and service robotics.

This project contributes to the field of autonomous robotics by demonstrating the implementation of autonomous robotics by demonstrating the implementation of the A* path planning algorithm within the MATLAB simulation environment. The developed system provides an interactive platform for generating collision-free paths, visualizing robot movement, and evaluating navigation performance through execution time and path length analysis. The interface (GUI), robot animation, and performance evaluation tools enhance the usability of the system and support a better understanding of autonomous navigation concepts.

In addition to its practical applications, the project has educational and research value. It provides a simulation platform for studying path planning algorithms, comparing navigation techniques, and analyzing robot performance under different environmental conditions. The development of system also serves as a foundation for future research,

including dynamic obstacle avoidance, sensor integration, real-world robot implementation, and advanced intelligent navigation methods.

1.9 Structure of the Report

This report is organized into seven chapters, each focusing on a specific phase of the project. The structure of the report is summarized in Table 1.1.

Table 1.1 Structure of the Report

CHAPTER	DESCRIPTION
1.Introduction	Introduces the background of autonomous mobile robots, path planning concepts, robot navigation, problem statement, project purpose, objectives, scope, importance, and report structure.
2.Literature Review	Reviews previous research related to autonomous mobile robots, path planning algorithms, MATLAB, graphical user interfaces, and existing navigation systems.
3.Research Methodology	Describes the research methodology, project development process, software development approach, and implementation methodology adopted for the project.
4.System Analysis and Design	Explains the system requirements, system architecture, UML diagrams (DFDs), use case diagrams, and overall system design.
5.Implementation	Present the implementation of the MATLAB GUI, A* path planning algorithm, robot animation, graph visualization, and algorithm comparison modules.
6.Results and Analysis	Discusses the simulation environment, experimental test cases, performance evaluation time, path length analysis, graphs, and discussion of results.
7.Conclusion and Future Work	Summarizes the project outcomes, achievements, limitations, and recommendations for future improvements.

The report follows a logical sequence, beginning with the project background and literature review, followed by the system development implementation process, implementation, performance evaluation, and concluding with the project's achievement opportunities. This structure provides a comprehensive understanding of the proposed Autonomous Mobile Robot Navigation System from Concept to evaluation.

2: Literature Review

2.1 Introduction

Autonomous mobile robot navigation has become one of the most important research areas in robotics due to its wide range of applications in industrial automation, warehouse logistics, healthcare, agriculture, and service robotics. The ability of a robot to navigate efficiently within an environment depends on the selection of an appropriate path planning algorithm and an effective navigation strategy.

Over the years, numerous path planning techniques have been developed to improve navigation efficiency, reduce computational cost, and ensure safe obstacle avoidance. These techniques include classical graph-based algorithms, heuristic search methods, and intelligent optimization approaches. Each method offers different advantages depending on the characteristics of the operating environment and application requirements.

This chapter reviews the fundamental concepts of autonomous mobile robots, commonly used robot path planning techniques, and several well-known path planning algorithms, including the Dijkstra algorithm, the A* algorithm, Rapidly Exploring Random Tree (RRT), and Genetic Algorithm-based path planning. The chapter also discusses the use of MATLAB as a simulation platform for robot navigation and presents a comparative analysis of the selected algorithms. This literature review provides the theoretical foundation for the development and implementation of the proposed autonomous mobile robot navigation system presented in the following chapters.

2.2 Autonomous Mobile Robots

An Autonomous Mobile Robot (AMR) is a robotic system capable of moving within an environment and performing assigned tasks without continuous human control. Unlike conventional mobile robots that follow predefined routes, AMRs can perceive their surroundings, analyze environmental information, and make navigation decisions autonomously. These capabilities enable AMRs to operate efficiently in structured and semi-structured environments while avoiding obstacles and adapting to changes within the workspace.

The operation of an autonomous mobile robot depends on several key components, including sensors, a control system, actuators, and a navigation algorithm. Sensors collect information about the surrounding environment, while the control system processes this information to determine appropriate movement decisions. The navigation algorithm calculates an efficient path from the starting point to the destination, and the actuators execute the required robot movements. The integration of these components enables safe and reliable autonomous navigation.

Autonomous mobile robots are widely used in industrial automation, warehouse logistics, healthcare, agriculture, and service robotics. Their ability to perform repetitive tasks with high accuracy improves operational efficiency, reduces human workload, and enhances workplace safety. As advances in robotics and artificial intelligence continue, autonomous mobile robots are becoming increasingly important in intelligent automation systems.

In this project, the autonomous mobile robot is simulated using the MATLAB environment, where the A* path planning algorithm is implemented to generate collision-free paths in a two-dimensional grid-based environment. The developed system provides an interactive platform for visualizing robot navigation and evaluating path planning performance.

2.3 Methods of Robot Path Planning

Robot path planning methods are used to determine a safe and efficient route from a starting position to a target destination while avoiding obstacles. Different path planning techniques have been developed to improve navigation performance based on the characteristics of the operating environment and application requirements. These methods can generally be classified into global path planning, local path planning, classical methods, heuristic-based techniques, and intelligent path planning approaches.

2.3.1 Classification of Path Planning Methods

The main path planning methods are summarized in Table 2.1.

Table 2.1 Classification of Robot Path Planning Methods

METHOD	DESCRIPTION	ADVANTAGES	LIMITATIONS
Global Path Planning	Generates a path using complete knowledge of the environment before navigation begins.	Produces optimal paths in known environments.	Cannot easily adapt to dynamic changes.
Local Path Planning	Determines navigation decisions based on real-time environment information.	Suitable for dynamic environments.	May produce non-optimal paths.
Classical Methods	Includes graph-based algorithms such as Dijkstra.	Simple, reliable, and guarantees shortest paths.	Computationally expensive in large environment.
Heuristic-Based Techniques	Uses heuristic functions to guide the search process, such as the A* algorithm.	Faster and more efficient than classical methods.	Performance depends on heuristic quality.
Intelligent Path Planning Techniques	Uses artificial intelligence and optimization methods, including Genetic Algorithms	Adaptive and suitable for complex optimization problems.	Higher computational complexity and longer execution time.

2.3.2 Importance of Path Planning Techniques

Path planning techniques play a vital role in autonomous robot navigation by enabling robots to reach their destinations safely and efficiently. An effective path planning algorithm minimizes travel distance, reduces execution time, avoids collisions, and improves overall navigation performance. The selection of an appropriate path planning method depends on factors such as environmental complexity, computational requirements, and application objectives.

In this project, the A* algorithm was selected because it provides an effective balance between computational efficiency and optimal path generation in grid-based environments, making it suitable for MATLAB-based autonomous mobile robot simulation.

2.4 Dijkstra Algorithm

The Dijkstra Algorithm is one of the most widely used graph-based path planning algorithms for finding the shortest path between a starting node and a destination node in a weighted graph. Developed by Edger W. Dijkstra in 1956, the algorithm systematically explores all possible paths and guarantees the shortest path when edge weights are non-negative

The algorithm begins by assigning zero to the starting node and infinity to all other nodes. It repeatedly selects the unvisited node with the smallest known distance, updates the distances of its neighboring nodes, and continues this process until the destination node is reached. This systematic approach ensures that the shortest path is identified.

Although the Dijkstra algorithm provides optimal path planning results, it evaluates many unnecessary nodes because it does not use heuristic information to guide the search process. As a result, its computational cost increases in large or complex environments, making it less efficient for real-time navigation applications.

Despite these limitations, the Dijkstra algorithm remains an important benchmark for evaluating the performance of other path planning methods. In this project, it is used as a reference algorithm for comparison with the A* algorithm to assess differences in execution time and navigation efficiency.

Table 2.2 Advantages and Limitations of the Dijkstra Algorithm

ADVANTAGES	LIMITATION
Guarantees the shortest path in weight graphs with non-negative edge weights.	Explores many unnecessary nodes, increasing computation time.
Simple and reliable graph search algorithm.	Less efficient in large search spaces.
Suitable for static and fully known environments.	Does not use heuristic information to guide the search process.

2.5 A* Algorithm

The A* (A-Star) algorithm is one of the most widely used heuristic-based path planning algorithms for autonomous mobile robot navigation. It is designed to determine the shortest and most efficient path between a starting point and a destination while avoiding obstacles. Due to its high search efficiency and ability to generate optimal paths, the A* algorithm has become a popular choice for robotics, game development, and autonomous navigation systems.

Unlike the Dijkstra algorithm, which explores nodes uniformly, the A* algorithm uses a heuristic function to guide the search process toward the target. It evaluates each node using the following evaluation function:

$$f(n)=g(n)+h(n)$$

Where:

- $g(n)$ represents the actual cost from the start node to the current node.
- $h(n)$ represents the estimated cost from the current node to the goal node.
- $f(n)$ represents the total estimated cost used to determine the next node for exploration.

By combining the actual path cost with a heuristic estimate, the A* algorithm reduces unnecessary node exploration and significantly improves search efficiency while maintaining optimal path generation.

The algorithm begins by placing the starting node in the Open List. During each iteration, the node with the lowest evaluation value is selected for expansion, and its neighboring nodes are evaluated. Once a node has been processed, it is moved to the Closed List to prevent repeated evaluation. This process continues until the destination node is reached, after which the optimal path is reconstructed by tracing the parent nodes.

The A* algorithm offers several advantages, including high search efficiency, optimal path generation, and relatively low computational cost in grid-based environments. However, its performance depends on the accuracy of the heuristic function and may require additional memory when applied to very large search spaces.

In this project, the A* algorithm is implemented within the MATLAB simulation environment to generate collision-free paths for an autonomous mobile robot. The generated path is visualized through the Graphical User Interface (GUI), and its performance is evaluated using execution time and path length. The algorithm is also compared with the Dijkstra algorithm to demonstrate its effectiveness in autonomous robot navigation.

Table 2.3 Advantages and Limitations of the A* Algorithm*

ADVANTAGES	LIMITATION
Generates optimal collision-free paths when an admissible heuristic is used.	Performance depends on the accuracy of the heuristic function.
Faster than traditional graph search algorithms such as Dijkstra.	Memory usage increases in very large environments.
Efficient for grid-based path planning.	Less suitable for highly dynamic environments without modification.
Widely used in robotics and autonomous navigation.	Requires heuristic design for effective performance.

2.6 Rapidly Exploring Random Tree (RRT)

The Rapidly Exploring Random Tree (RRT) algorithm is a sampling-based path planning technique developed for robot navigation in complex and high-dimensional environments. Unlike graph-based algorithms, RRT explores the search space by generating random sample points and gradually expanding a tree structure from the starting position toward the target location. This approach enables the algorithm to efficiently discover feasible paths without requiring complete knowledge of the environment.

The RRT algorithm begins by initializing a tree with the starting node. During each iteration, a random point is generated within the search space, and the nearest existing node in the tree is identified. A new node is then created by extending the tree toward the random point while ensuring that the generated path is collision-free. This process continues until the tree reaches its destination or satisfies the predefined termination condition.

One of the major advantages of RRT is its ability to explore large and complex environments efficiently, making it suitable for high-dimensional motion planning problems. It is particularly effective in environments where traditional graph-based algorithms become computationally expensive. However, the paths generated by RRT are not always optimal and often require additional path smoothing or optimization techniques to improve navigation quality.

Although the RRT algorithm is widely used in robotic motion planning, this project adopts the A* algorithm because it provides optimal path generation, lower computational complexity in grid-based environments, and better suitability for MATLAB-based autonomous mobile robot simulation.

Table 2.4 Advantages and Limitations of the RRT Algorithm

ADVANTAGES	LIMITATIONS
Efficient for large and high-dimensional search spaces.	Does not always generate the shortest path.
Suitable for complex environments.	Generated paths may require smoothing or optimization.
Can rapidly explore unknown search spaces.	Performance depends on random sampling.
Widely used in robotic motion planning.	Less suitable than A* for structured Grid-based environments.

2.7 Genetic Algorithm-Based Path Planning

The Genetic Algorithm (GA) is an optimization-based path planning technique inspired by the principles of natural selection and biological evolution. It is widely used to solve complex optimization problems by generating multiple candidate solutions and gradually improving them through iterative evolutionary processes. In robot path planning, the Genetic Algorithm searches for a feasible and efficient path by optimizing predefined evaluation criteria such as path length, obstacle avoidance, and navigation cost.

The algorithm begins by generating an initial population of possible paths. Each path is evaluated using a fitness function that measures its quality based on the selected optimization objectives. The best-performing paths are then selected to produce new solutions through genetic operations such as selection, crossover, and mutation. This iterative process continues until an acceptable solution, or the optimal path is obtained.

Genetic Algorithms are highly flexible and can efficiently solve complex path planning problems involving multiple optimization objectives. They are particularly suitable for large search spaces and environments where traditional deterministic algorithms may not perform effectively. However, the quality of the generated solution depends on the design of the fitness function and algorithm parameters. In addition, Genetic Algorithms generally require higher computational time than heuristic-based algorithms because several generations must be evaluated before reaching an optimal solution.

Although the Genetic Algorithm is a powerful optimization technique, this project employs the A* algorithm because it provides faster path generation, lower computational cost, and guaranteed optimal paths in grid-based environments. These characteristics make the A* algorithm more suitable for the MATLAB-based autonomous mobile robot navigation system developed in this project.

Table 2.5 Advantages and Limitations of the Genetic Algorithm

ADVANTAGES	LIMITATIONS
Effective for solving complex optimization problems.	Requires higher computational time than heuristic algorithms.
Can optimize multiple objectives simultaneously.	Performance depends on fitness function and parameter selection.
Suitable for large and complex search spaces.	Does not always guarantee the global optimal solution.
Flexible and adaptable to different applications.	May require many generations to converge.

2.8 MATLAB for Robot Simulation

MATLAB is a high-level programming and numerical computing environment developed by MathWorks for scientific computing, data analysis, algorithm development, and simulation. Due to its powerful mathematical functions, visualization capabilities, and built-in development tools, MATLAB has become one of the most widely used software platforms for robotics research and education.

In robot simulation, MATLAB provides an efficient environment for developing, testing, and evaluating path planning algorithms before real-world implementation. Its graphical visualization capabilities enable researchers to create simulation environments, model robot movement, and analyze navigation performance without requiring physical robotic hardware. This reduces development costs and allows algorithms to be evaluated under different experimental conditions.

One of the major advantages of MATLAB is its support for Graphical User Interface (GUI) development, which allows users to interact with simulation models through an intuitive interface. MATLAB also provides extensive libraries and toolboxes that simplify algorithm implementation, performance analysis, and graphical representation of simulation results.

In this project, MATLAB R2014b is used as the development platform to implement the A* path planning algorithm, simulate autonomous robot navigation, develop the graphical user interface, visualize robot movement, generate performance graphs, and compare the A* algorithm with the Dijkstra algorithm. The flexibility and reliability of MATLAB make it an appropriate platform for developing and evaluating the proposed autonomous mobile robot navigation system.

Table 2.6 Advantages of MATLAB for Robot Simulation

FEATURE	CONTRIBUTION TO THE PROJECT
Numerical Computing	Supports efficient implementation of path planning algorithms.
GUI Development	Enables the creation of an interactive user interface for robot navigation.
Graphics and Visualization	Displays robot movement, navigation paths, and performance graphs.
Simulation Environment	Allows testing without physical robotic hardware.
Performance Analysis	Supports execution time measurement and path length evaluation.

2.9 Comparison of Path Planning Algorithms

Various path planning algorithms have been developed to improve the navigation performance of autonomous mobile robots. Each algorithm differs in terms of search strategy, computational efficiency, path optimality, and suitability for different operating environments. Therefore, selecting an appropriate algorithm depends on the application requirements, environmental complexity, and computational constraints.

Table 2.7 presents a comparison of the path planning algorithms discussed in this chapter based on their key characteristics.

Table 2.7 Comparison of Path Planning Algorithms

ALGORITHM	PATH OPTIMALITY	SEARCH SPEED	SUITABLE ENVIRONMENT	ADVANTAGES	LIMITATIONS
Dijkstra	Optimal	Moderate	Static environment	Guarantees the shortest path & is simple to implement.	Explores many unnecessary nodes, increasing computation time.
A*	Optimal	High	Static grid-based environment	Fast search using heuristic guidance with efficient path generation.	Performance depends on the heuristic function.
RRT	Feasible (Not always optimal)	High	Large and complex environments	Efficiently explores high-dimensional search spaces.	Generated paths often require smoothing and optimization.
Genetic Algorithm	Near Optimal	Moderate	Complex optimization problems	Flexible and capable of handling multiple optimization objectives,	Higher computational cost and longer convergence time.

The comparison indicates that each algorithm has its own strengths and limitations. Dijkstra provides reliable shortest-path solutions but requires more computational effort because it evaluates many nodes. RRT is effective in complex environments but does not guarantee an optimal path, while the Genetic Algorithm offers flexibility for optimization problems at the expense of increased computational time.

Considering the objectives of this project, the A* algorithm was selected because it offers an effective balance between computational efficiency and optimal path generation. Its heuristic-based search strategy enables faster navigation while maintaining collision-free and optimal paths, making it well suited for implementation in the MATLAB-based autonomous mobile robot navigation system.

2.10 Summary of Literature Review

This chapter reviewed the fundamental concepts related to autonomous mobile robot navigation and the path planning techniques used in intelligent robotic systems. The literature highlighted the characteristics of autonomous mobile robots, the classification of robot path planning methods, and the principles of several widely used algorithms, including the Dijkstra algorithm, the A* algorithm, Rapidly Exploring Random Tree (RRT), and Genetic Algorithm-based path planning.

The review also examined the role of MATLAB as an effective simulation platform for developing, testing, and evaluating robot navigation systems. In addition, a comparative analysis of the selected path planning algorithms demonstrated that each method has specific advantages and limitations depending on the application requirements and environmental conditions.

Based on the literature reviewed, the A* algorithm was identified as the most suitable path planning technique for this project due to its ability to generate optimal collision-free paths with high computational efficiency in grid-based environments. The theoretical concepts and findings presented in this chapter provide the foundation for the research methodology, system design, and implementation discussed in the subsequent chapters.

3: Methodology

3.1 System Overview

The Autonomous Mobile Robot Path Planning System was developed to demonstrate the implementation of intelligent navigation techniques in a simulated robotic environment. The system utilizes the A* Algorithm to generate an optimal collision-free path between a user-defined start location and goal location while avoiding obstacles present within the environment.

The proposed system is implemented using MATLAB and consists of a graphical user interface (GUI) that allows users to interact with the simulation environment. Through the GUI, users can create a map, define start and goal positions, execute the path planning algorithm, visualize robot movement, and analyze system performance.

The overall operation of the system begins with the creation of a two-dimensional grid-based environment. Obstacles are then placed within the environment to simulate real-world navigation constraints. After selecting the start and goal positions, the A* Algorithm is executed to calculate the shortest feasible path.

3.2 Project Development Process

The development of the Autonomous Mobile Robot Path Planning System was carried out through a series of systematic stages to ensure successful implementation, testing, and performance evaluation. Each stage focused on a specific component of the system, gradually transforming the project from a basic simulation model into a complete path planning and analysis platform.

Table 3.1 Project Development Process

STAGE	DESCRIPTION
1	Environment Design
2	GUI Development
3	A* Implementation
4	Path Visualization
5	Robot Animation
6	Performance Evaluation
7	Graph Analysis
8	Algorithm Comparison
9	Testing & Validation

Through these development stages, the project evolved into a comprehensive autonomous mobile robot path planning platform capable of simulation, visualization, performance evaluation, and algorithm comparison.

3.3 Project Workflow

The workflow of the Autonomous Mobile Robot Path Planning System describes the sequence of operations performed by the system from environment creation to navigation performance analysis. The workflow ensures that all stages of path planning, simulation, and evaluation are executed in a structured and logical manner.

To further support analysis, graphical visualization modules display performance results through charts and plots. Additionally, the algorithm comparison module compares the execution times of the A* Algorithm and the Dijkstra Algorithm to evaluate computational efficiency.

The complete workflow of the system can be summarized as follows:

- Create simulation environment.
- Generate obstacle-based maps.
- Select start position.
- Select goal position.
- Execute A* Algorithm.
- Generate optimal path and Visualize path on map.
- Animate robot movement and Calculate path length.
- Measure execution time and Display performance graphs.
- Compare A* and Dijkstra algorithms and Analyze results and performance.

3.4 Environment Modeling

Environment modeling is a critical component of autonomous mobile robot navigation because it provides the workspace in which the robot performs path planning and movement operations. In this project, a two-dimensional grid-based environment was developed using MATLAB to simulate a realistic navigation scenario for the autonomous mobile robot.

The simulation environment consists of a matrix structure where each cell represents a specific location within the workspace. The grid-based representation simplifies obstacle detection, path generation, and navigation analysis while providing an efficient platform for implementing the A* Algorithm.

Within the modeled environment, cells are classified into different categories:

- Free cells represent navigable areas where the robot can move freely.
- Obstacle cells represent blocked regions that cannot be crossed by robots.
- Start cells indicate the robot's initial position.
- Goal cells indicate the target destination.
- Path cells represent the optimal route generated by the A* Algorithm.

The environment is displayed graphically using MATLAB visualization functions. Different colors and symbols are used to distinguish between obstacles, navigation points, and generated paths, improving the readability of the simulation.

3.5 Obstacle Generation

Obstacle generation is an important aspect of robot path planning because obstacles create navigation constraints that the robot must avoid while moving toward its destination. In autonomous navigation systems, obstacle representation enables the evaluation of path planning algorithms under realistic operating conditions.

In this project, obstacles are represented within the grid-based environment using specific matrix values. Cells containing obstacle values are considered non-navigable regions and cannot be crossed by the robot during path planning. These obstacle cells force the algorithm to search for alternative routes and generate collision-free paths.

When the A* Algorithm is executed, obstacle cells are automatically excluded from the search process. The algorithm evaluates only valid neighboring cells that are free from obstacles. This ensures that the generated path does not intersect with blocked regions and remains safe for robot navigation.

Graphical visualization techniques are used to distinguish obstacle cells from free navigation cells. Obstacles are displayed using dedicated colors within the simulation environment, allowing users to easily identify restricted areas and observe how the generated path avoids them.

3.6 Start and Goal Selection

The start and goal selection process is a fundamental step in the autonomous navigation workflow because it defines the robot's initial location and intended destination. These two positions serve as the primary inputs for the A* path planning algorithm and determine the navigation route generated by the system.

In the developed MATLAB graphical user interface, dedicated buttons are provided to allow users to interactively select the start position and goal position within the simulation environment. This interactive approach improves usability and enables users to create different navigation scenarios without modifying the source code.

Start Position Selection

The start position represents the initial location of the autonomous mobile robot within the grid-based environment. When the Select Start button is activated, the user can click on a valid location within the simulation map to specify the robot's starting point.

Goal Position Selection

The goal position represents the target destination that the robot must reach through autonomous navigation. After selecting the start position, the user activates the Select Goal button and chooses a destination point within the environment.

Validation of Selected Positions

To ensure proper navigation, the system verifies that both the start position and goal position are located within valid navigable cells of the environment. Positions that overlap with obstacle cells are not considered suitable navigation points because they would prevent successful path generation.

The path planning process cannot begin until both the start and goal positions have been successfully defined. If either position is missing, the system generates an appropriate message requesting the user to complete the selection process.

3.7 A* Algorithm Implementation

The A* Algorithm serves as the core path planning mechanism of the proposed Autonomous Mobile Robot Navigation System. The algorithm is responsible for generating the shortest collision-free path between the selected start position and goal position while avoiding obstacles within the simulation environment.

The implementation of the A* Algorithm was carried out using MATLAB. The algorithm operates on the grid-based environment created during the environment modeling stage and utilizes graph search principles to identify an optimal navigation route.

Working Principle of the A* Algorithm

The evaluation function is defined as:

$$[f(n) = g(n) + h(n)]$$

where:

$f(n)$ = Total estimated cost

$g(n)$ = Actual cost from the start node to the current node

$h(n)$ = Estimated cost from the current node to the goal node

3.7.1 Open List

The Open List contains nodes that have been discovered but not yet explored. During each iteration, the node with the smallest evaluation cost is selected for further processing.

3.7.2 Closed List

The Closed List contains nodes that have already been explored. Once a node has been processed, it is transferred to the Closed List to prevent repeated evaluation.

3.7.3 Cost Function

The cost function determines the priority of nodes during the search process. By combining actual travel cost and estimated future cost, the algorithm balances path optimality and search efficiency.

3.7.4 Heuristic Function

The heuristic function estimates the remaining distance between the current node and the goal node. In this project, distance-based heuristic calculations are used to guide the search process toward the target location.

Algorithm Execution Process

The implementation follows following sequence:

- Initialize the Open List with the start node.
- Calculate cost values for neighboring nodes.
- Select the node with the lowest $f(n)$ value.
- Move the selected node to the Closed List.
- Generate valid neighboring nodes.
- Ignore obstacle cells and previously explored nodes.
- Update node costs when shorter paths are found.
- Continue the search until the goal node is reached.
- Reconstruct the optimal path from goal to start.

Application within the Proposed System

The resulting path is subsequently used for performance evaluation, path length analysis, execution time measurement, graphical visualization, and comparison with the Dijkstra Algorithm. Therefore, the A* Algorithm forms the foundation of the entire autonomous navigation system developed in this project.

3.8 Path Generation Process

The path generation process is one of the most important operations in the Autonomous Mobile Robot Path Planning System. Its primary objective is to identify an optimal collision-free route that enables the robot to travel safely from the selected starting position to the goal position while avoiding obstacles within the environment.

In this project, path generation is performed using the A* Algorithm. Once the user selects the start and goal locations and initiates the simulation, the algorithm begins analyzing the grid-based environment to determine a suitable navigation route.

As the search progresses, the algorithm continuously explores valid neighboring nodes and updates path information whenever a shorter route is discovered. This process continues until the destination node is successfully reached.

After reaching the goal node, the algorithm reconstructs the navigation route by tracing the parent nodes from the destination back to the starting position. This reconstruction process produces the final optimal path used for robot navigation.

Path Generation Procedure

The path generation procedure consists of the following steps:

- Initialize the start node.
- Add the start node to the Open List.
- Select the node with the minimum cost value.
- Generate neighboring nodes.
- Check for obstacle-free movement.
- Calculate path costs.
- Update node information when better paths are found.
- Continue the search process.
- Reach the goal node.
- Reconstruct the final path.
- Display the generated route.

Path Visualization

Once the path is generated, it is displayed graphically within the simulation environment. The path is highlighted using a dedicated visual representation, allowing users to clearly observe the route selected by the algorithm.

3.9 Robot Animation Process

Robot animation is implemented to provide a visual representation of the navigation process within the simulation environment. The animation enables users to observe how the autonomous mobile robot follows the generated path from the start position to the goal position while avoiding obstacles.

After path generation, the robot moves step by step along the calculated path until it reaches the goal. The robot is then moved sequentially through each node of the path until it reaches its destination.

The animation process improves the visualization capability of the system and allows users to better understand the robot's navigation behavior.

Animation Procedure

The robot animation process follows the steps below:

- Generate the optimal path using the A* Algorithm.
- Store the path coordinates.
- Initialize the robot position at the start node.
- Display the robot on the simulation map.
- Move the robot along the calculated path.
- Update the robot position at each step.
- Refresh the graphical display continuously.
- Continue movement until the goal node is reached.
- Stop the animation at the destination.

Visualization Method

The robot is represented graphically within the MATLAB environment using markers and path visualization techniques. The generated route is displayed as a highlighted path while the robot moves step-by-step along the selected trajectory.

Different colors are used to improve visualization:

- Green Marker – Start Position
- Red Marker – Goal Position
- Blue Line – Generated Path
- Yellow Marker – Moving Robot

3.10 Performance Evaluation Method

Performance evaluation is conducted to measure the effectiveness and efficiency of the developed Autonomous Mobile Robot Path Planning System. The evaluation process focuses on assessing the quality of the generated path and the computational performance of the implemented path planning algorithm.

Several performance metrics are used to analyze the behavior of the system under different simulation conditions. These metrics provide quantitative evidence regarding the capability of the proposed navigation system.

Test Environment

The system was evaluated using multiple obstacle configurations in the MATLAB simulation environments. For each test case, different obstacle configurations are created while maintaining valid start and goal locations.

The experiments are conducted within the MATLAB simulation environment using the developed graphical user interface.

Path Length Measurement

Path length is one of the primary performance indicators used in this project. It represents the total distance traveled by the robot from the start position to the goal position.

A shorter path generally indicates better navigation efficiency because the robot can reach its destination using less movement and reduced travel time.

The path length value is automatically calculated and displayed within the MATLAB GUI after each simulation.

Execution Time Measurement

Execution time measures the computational efficiency of the path planning algorithm. It represents the time required by the algorithm to calculate the optimal route.

MATLAB timing functions are used to record the execution duration of each simulation run. Lower execution times indicate higher algorithm efficiency and faster decision-making capability.

Multiple Test Case Evaluation

To ensure reliable performance analysis, the system is evaluated using multiple test cases. Multiple test cases with different obstacle configurations were used to evaluate the algorithm

Graph-Based Performance Analysis

Graphical visualization techniques are employed to present performance results clearly.

The developed system includes:

- Path Length Analysis Graph
- Execution Time Comparison Graph
- Algorithm Comparison Graph

These visualizations simplify the interpretation of experimental results and assist in identifying performance trends.

A* and Dijkstra Algorithm Comparison

An additional performance evaluation is conducted by comparing the A* Algorithm with the Dijkstra Algorithm.

The comparison focuses on:

- Execution Time
- Search Efficiency
- Computational Performance

Experimental results demonstrate that the A* Algorithm generally requires less execution time because the heuristic function guides the search toward the goal more efficiently.

Evaluation Metrics Summary

The following metrics are used throughout the evaluation process:

Table 3.2 Evaluation Metrics Summary

PERFORMANCE METRIC	DESCRIPTION
Path Length	Total distance traveled by the robot
Execution Time	Time required to generate the path
Number of Test Cases	Total simulation experiments performed
Algorithm Efficiency	Performance comparison between A* and Dijkstra
Graphical Analysis	Visualization of performance results

3.11 Methodology Flowchart

A methodology flowchart is used to illustrate the overall operational sequence of the proposed Autonomous Mobile Robot Path Planning System. The flowchart provides a clear visual representation of the processes involved in map creation, path planning, robot navigation, and performance evaluation. The developed system follows a systematic workflow beginning with environment generation and ending with result visualization and analysis.

Methodology Flowchart

START



Launch MATLAB GUI



Create Random Environment



Generate Obstacles



Select Start Position



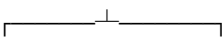
Select Goal Position



Run A* Algorithm



Path Found?



NO



YES



Display Error Generate Path

Message



Animate Robot



Calculate Path Length



Calculate Execution Time



Display Performance Results



Show Graphs and Algorithm Comparison ____ END

4: System Design

4.1 System Architecture

System architecture describes the overall structure of the Autonomous Mobile Robot Path Planning System and illustrates how different components interact to achieve autonomous navigation. The architecture integrates user interaction, environment modeling, path planning, robot animation, and performance evaluation within a unified MATLAB simulation platform.

The proposed system follows a modular architecture in which each functional component performs a specific task while communicating with other modules. This design approach improves maintainability, scalability, and system organization.

Architecture Components

The system consists of the following major components:

Environment Modeling Module

The environment modeling module is responsible for generating the grid-based navigation workspace. It creates a simulation map and manages obstacle placement within the environment.

Path Planning Module

The path planning module contains the implementation of the A* Algorithm. This module implements the A* algorithm to generate the optimal collision-free path

This module acts as the core decision-making component of the system.

Robot Animation Module

After path generation, the robot animation module visualizes robot movement along the calculated route.

Performance Evaluation Module

The performance evaluation module calculates navigation metrics. This module measures path length and execution time.

These measurements are used for system performance analysis.

Graph Visualization Module

The graph visualization module presents experimental results using graphical representations.

It enables users to analyze:

- 1.Path Length Trends
2. Performance Results
3. Navigation Statistics

Algorithm Comparison Module

An additional comparison module was implemented to evaluate the performance of different path planning techniques.

The module compares:

- A* Algorithm
- Dijkstra Algorithm

The comparison is based on execution time and computational efficiency.

The proposed architecture provides a reliable framework for autonomous robot navigation simulation and serves as the foundation for all system operations performed within the project.

4.2 GUI Design

The Graphical User Interface (GUI) was developed to provide an interactive and user-friendly environment for operating the Autonomous Mobile Robot Path Planning System. The GUI was implemented using MATLAB and serves as the primary interface through which users interact with the simulation environment.

The design objective was to create a simple, intuitive, and efficient interface that enables users to perform path planning operations without requiring direct interaction with the source code.

GUI Layout

The GUI consists of two main sections:

- 1.Simulation Environment Display Area
- 2.Control Panel

The simulation environment display area is responsible for visualizing the navigation map, obstacles, start position, goal position, generated path, and robot movement.

The control panel contains buttons and controls used to perform system operations.

Performance Information Display

The GUI also includes performance indicators that display:

- Path Length
- Execution Time

These values are automatically updated after each simulation run and provide immediate feedback regarding navigation performance.

GUI Components

The developed GUI includes the following functional buttons:

Table 4.1 GUI Components

GUI COMPONENT	FUNCTION
Create Map	Generates a grid-based environment with obstacles.
Select Start	Select the robot's starting position.
Select Goal	Select the destination position.
Run Simulation	Executes the A* algorithm and animates the robot.
Reset	Clears the simulation environment.
Show Graph	Displays performance graphs.
Compare Algorithms	Compares A* and Dijkstra performance.

User Interaction Process

The user interacts with the system using the following sequence:

- Create the simulation map.
- Select the start position.
- Select the goal position.
- Run the simulation.
- Observe path generation and robot movement.
- Analyze performance results.
- View graphs and algorithm comparisons.

Advantages of GUI Design

The developed GUI offers several advantages:

- User-friendly operation.
- Interactive simulation environment.
- Clear visualization of navigation processes.
- Real-time performance feedback.
- Simplified experimentation and testing.
- Professional presentation of simulation results.

GUI serves as the central interaction platform of the proposed system and significantly enhances the usability, visualization, and analysis capabilities of the Autonomous Mobile Robot Path Planning System.

4.3 Functional Modules

The Autonomous Mobile Robot Path Planning System is organized into several functional modules, each responsible for a specific task within the navigation process. The modular architecture improves maintainability, simplifies debugging, and supports future system enhancements. The main functional modules are summarized in Table 4.1.

Table 4.2 Functional Modules of the Proposed System

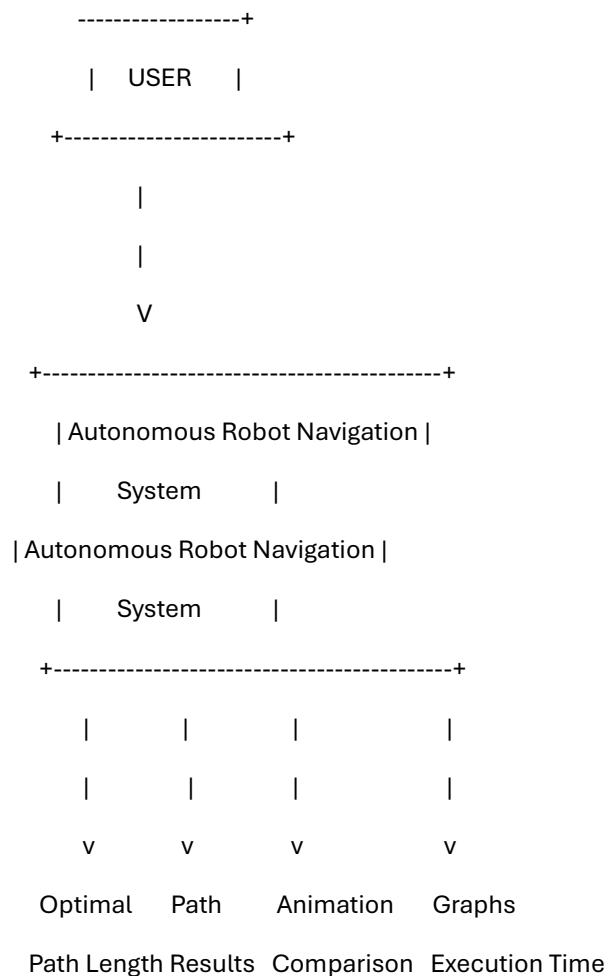
MODULE	FUNCTION
Create Map	Generate the grid-based simulation environment and obstacle configuration.
Select start	Allow the user to choose the robot's starting position.
Select Goal	Allow the user to define the destination position.
Run Simulation	Executes the A* algorithm, generates the optimal path, and starts robot animation.
Path Planning	Calculates the collision-free shortest path using the A* algorithm.
Robot Animation	Animates the robot movement along the generated path.
Performance Evaluation	Calculates path length and execution time after each simulation
Show Graph	Display graphs for performance analysis.
Compare Algorithms	Compares the execution performance of the A* algorithms.
Reset	Clears the current simulation and prepares the system for a new experiment.

All modules operate together to complete the navigation process. The Create Map module initializes the environment, the Start and Goal modules collect user input, the Path Planning module generates the optimal route, the Robot Animation module visualizes robot movement, and the evaluation modules analyze navigation performance. This modular design improves software organization while providing a flexible platform for autonomous mobile robot simulation.

4.4 Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) illustrates how information moves between the user, the graphical user interface (GUI), the processing modules, and the output components of the proposed Autonomous Mobile Robot Path Planning System. It provides a simplified representation of the system's data processing activities without describing implementation details.

Level 0 DFD Structure



The navigation process begins when the user creates the simulation environment and selects the start and goal positions through GUI. These inputs are transferred to the path planning module, where the A* algorithm computes the optimal collision-free path. The generated path is then forwarded to the robot animation module for visualization while the performance evaluation module calculates the execution time and path length. Finally, the processed results are displayed as graphs and algorithm comparison outputs through the graphical interface.

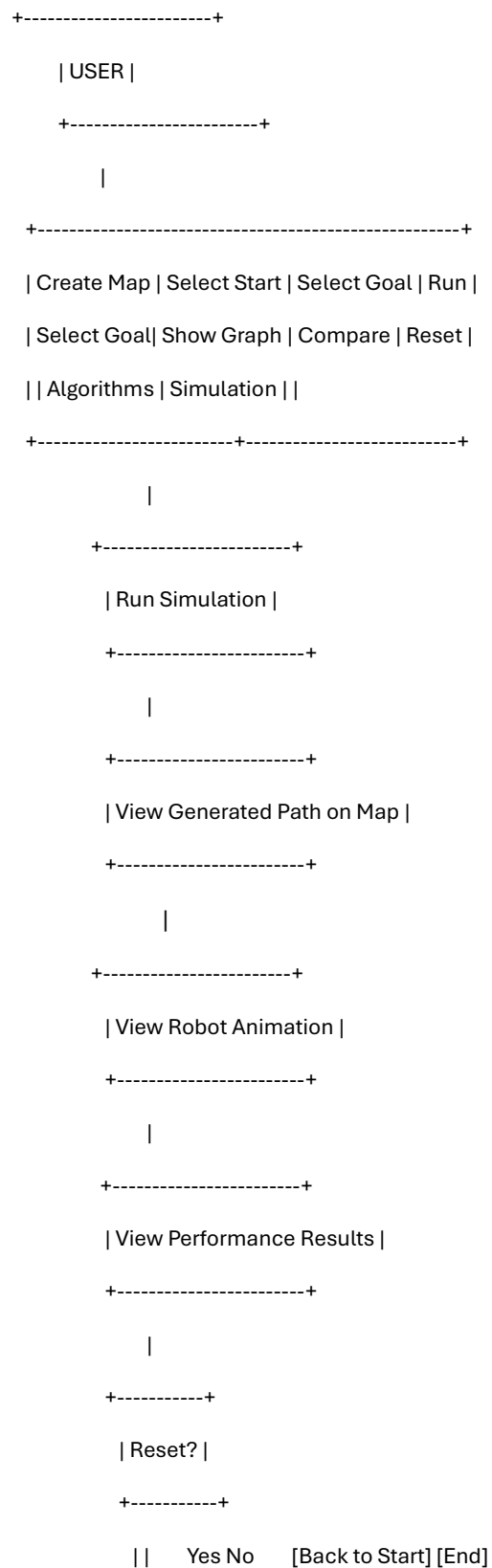
The DFD demonstrates the interaction between the major functional modules and shows how user inputs are transformed into navigation results. It also highlights the relationship between the path planning, visualization, and performance evaluation components within the proposed system.

4.5 Use Case Diagram

The Use Case Diagram illustrates the interactions between the user and the proposed Autonomous Mobile Robot Path Planning System. It identifies the primary system functions available through the MATLAB graphical user interface (GUI) and shows how

the user interacts with each function to perform navigation and performance evaluation tasks.

Use Case Diagram Structure



The user initiates the simulation by creating the environment and selecting the start and goal positions. The system then executes the A* algorithm to generate the optimal collision-free path and animates the robot along the calculated route. After the simulation, the user can view performance graphs, compare the A* and Dijkstra algorithms, and reset the environment for additional experiments.

The Use Case Diagram provides a high-level overview of the functional requirements and demonstrates how the user interacts with the major components of the proposed navigation system.

5: Implementation

5.1 Software Requirements

MATLAB Software

MATLAB was selected as the development platform due to its powerful numerical computing capabilities, integrated development environment, and built-in visualization tools. It provides a suitable environment for implementing the A* path planning algorithm, developing the graphical user interface, and evaluating system performance.

MATLAB GUI Components

Table 5.1 Software Components

COMPONENT	FUNCTION
Axes	Display the simulation environment
Push Buttons	Execute system functions
Labels	Display system information
Plot Area	Show performance graphs

A* Path Planning Implementation

The A* Algorithm was implemented using MATLAB programming functions and data structures.

The implementation required:

- Node management
- Cost calculations
- Open List processing
- Closed List processing
- Path reconstruction

These functions enabled efficient path generation within the simulation environment.

Graphical Visualization Functions (Table 5.2)

FUNCTION	PURPOSE
Plot	Display navigation path
Fill	Draw obstacles
Animation	Move robot
Graph	Display performance analysis

Hardware Requirements

The implementation can be executed on a standard personal computer with the following minimum specifications: (Table 5.3)

COMPONENT	REQUIREMENT
Processor	Intel Core i3 or Higher
RAM	4GB or Higher
Storage	2GB Free Space
Operating System	Windows 7/8/10/11
MATLAB Version	MATLAB R2014b or Later

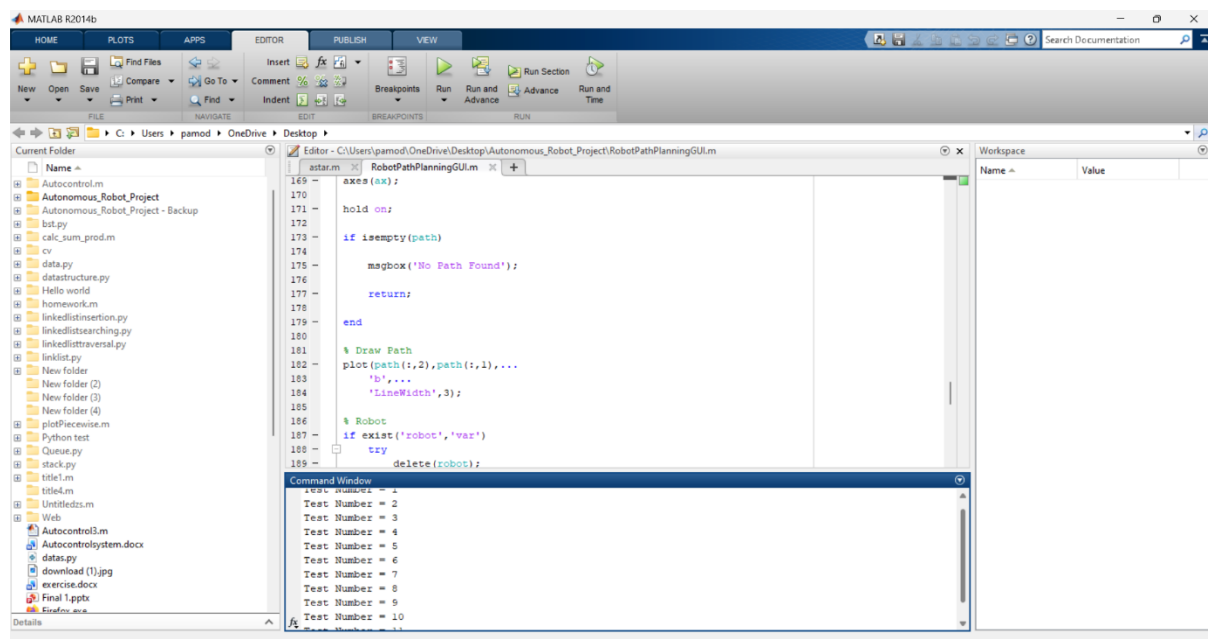
Development Tools Summary (Table 5.4)

SOFTWARE TOOL	PURPOSE
MATLAB	System Development
MATLAB GUI	User Interface design
MATLAB Plot Functions	Graph Visualization
MATLAB Timing Function	Performance Evaluation

5.2 MATLAB Development Environment

The proposed Autonomous Mobile Robot Path Planning System was developed using MATLAB R2014b, which provides an integrated development environment (IDE) for algorithm implementation, graphical user interface (GUI) development, data visualization, and performance analysis. The environment supports rapid application development through its built-in numerical computation and visualization capabilities, making it suitable for autonomous robot simulation.

Figure 5.1 MATLAB R2014b Development Environment



MATLAB R2014b Development Features

MATLAB R2014b offers a comprehensive development environment that simplifies software development mentation and testing. The main features used in this project are summarized in Table 5.2.

Table 5.5 MATLAB Development Features

FEATURE	PURPOSE
MATLAB Editor	Develop and modify program code
Command Window	Execute and debug MATLAB commands
Figure Window	Displaying the environment
Plot Functions	Generate performance graphs
GUI Components	Provide user interaction through the interface

Visualization Functions

MATLAB provides built-in graphical functions for displaying the simulation environment, robot movement, and performance results. These functions were used to visualize the generated path, animate robot navigation, and present execution time and path length using graphs.

Development Workflow

The MATLAB development process followed the sequence below:

- Environment Creation
- GUI Development
- A* Algorithm Implementation
- Path Generation
- Robot Animation
- Performance Evaluation
- Graph Visualization
- Algorithm Comparison

This structured workflow enabled systematic development and testing of the proposed system.

Project Compatibility

The developed project was designed to operate successfully within MATLAB R2014b while maintaining compatibility with newer MATLAB versions. This ensures that the simulation can be executed on a wide range of MATLAB installations without requiring major modifications.

5.3 Project Structure

The proposed Autonomous Mobile Robot Path Planning System was developed using a modular software architecture to improve code organization, maintainability, and future scalability. Each module performs a specific function while interacting with other modules to complete the navigation process. The project is implemented through the main program file, RobotPathPlanningGUI.m, which coordinates all system operations.

Main Project File

The RobotPathPlanningGUI.m file serves as the main application of the system. It initializes the graphical user interface (GUI), manages user interactions, executes the A* path planning algorithm, controls robot animation, and coordinates the performance evaluation process.

Project Module Structure

The major software modules used in the project are summarized in Table 5.3.

Table 5.6 Project Module Structure

MODULE	FUNCTION
GUI Module	Provide the graphical interface and manages user interactions.
Environment Module	Creates the grid-based environment and obstacle configuration.
A* Path Planning Module	Generate the optimal collision-free path between the selected start and goal positions.
Robot Animation Module	Animates robot movement along the generated path.
Performance Evaluation Module	Calculates execution time and path length after each simulation
Graph Visualization Module	Displays graphs for navigation performance analysis.
Algorithm Comparison Module	Compare the performance of the A* and Dijkstra algorithms.

Logical Project Structure

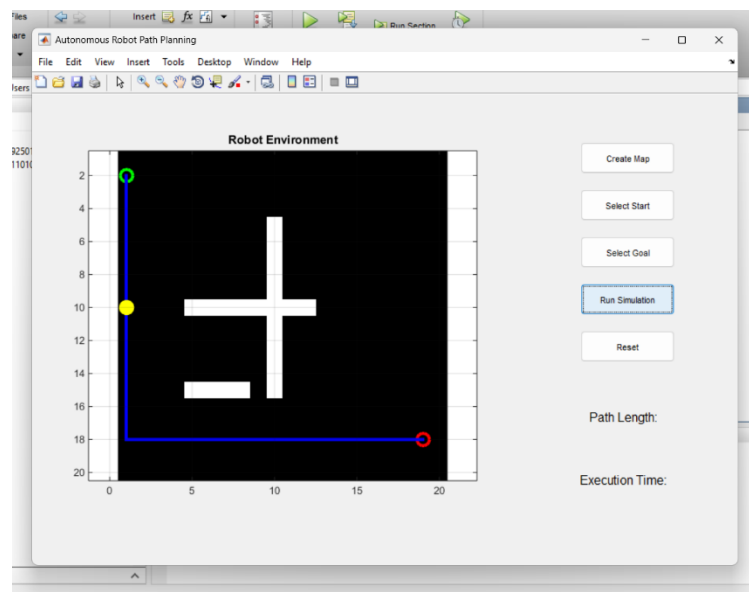
The system operates through sequential workflow. After the user creates the simulation environment and selects the start and goal positions, the A* path planning module generates the optimal path. The robot animation module then visualizes the robot movement, while the performance evaluation and graph visualization modules analyze and present the simulation results. Finally, the algorithm comparison module evaluates the performance differences between the implemented algorithms.

The modular project structure improves software organization by separating individual system functions into independent components. This approach simplifies development, debugging, testing, and future enhancements while maintaining efficient interaction between all modules.

5.4 GUI Implementation

The Graphical User Interface (GUI) was implemented using MATLAB to provide an interactive platform for autonomous mobile robot path planning and performance evaluation. GUI integrates all major system functions into a single interface, allowing users to create a simulation environment, execute the A* algorithm, visualize robot movement, and analyze performance results.

Figure 5.2 Main Graphical User Interface



Main GUI Components

The main components implemented in the GUI are summarized in Table 5.4.

Table 5.7 Main GUI Components

GUI COMPONENT	FUNCTION
Simulation Area	Displays the grid environment, obstacles, robots, and generated path.
Create Map Button	Generates the simulation environment with obstacles.
Select Start Button	Allow the user to select the robot's starting position.
Select Goal Button	Allow the user to define the destination position.
Run Simulation Button	Executes the A* algorithm and starts robot navigation.
Reset Button	Clears the current simulation and prepares the environment for a new test.
Show Graph Button	Display execution time and path length graphs.
Compare Algorithm Button	Compare the performance of the A* and Dijkstra algorithms.
Status Display	Displays system messages and simulation status.

Callback Function Implementation

Each GUI component is associated with a callback function that responds to user actions. These callback functions manage environment creation, start and goal selection, algorithm execution, robot animation, performance evaluation, and graph visualization, ensuring smooth interaction between the user interface and the underlying processing modules.

GUI Operation Sequence

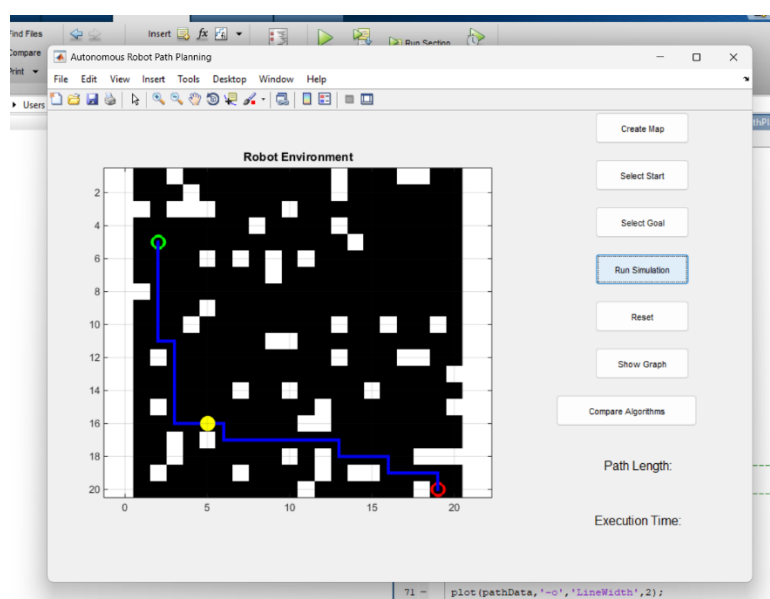
The user first creates the simulation environment and selects the start and goal positions. The A* algorithm is then executed to generate the optimal path, after which the robot movement is animated. Finally, performance graphs and algorithm comparison results are displayed, while the reset function allows the simulation to be restarted for additional experiments.

The implemented GUI provides an integrated and user-friendly environment for autonomous robot navigation by combining visualization, interaction, and performance evaluation within a single application.

5.5 A* Algorithm Implementation

The A* path planning algorithm was implemented as the core navigation algorithm of the proposed Autonomous Mobile Robot Path Planning System. The implementation is responsible for generating the shortest collision-free path between the selected start and goal positions while avoiding obstacles within the grid-based environment.

Figure 5.3 Collision-Free Path Generated Using A* Algorithm



Implementation Components

The major functions implemented in the A* algorithm are summarized in Table 5.5.

Table 5.8 A* Algorithm Implementation Components*

COMPONENT	FUNCTION
Open List	Stores candidate nodes that are available for exploration.
Closed List	Stores evaluated nodes to prevent repeated processing.
Neighbor Node Evaluation	Calculates the movement cost of adjacent nodes.
Heuristic Function	Estimates the remaining distance from each node to the goal.
Path Reconstruction	Generates the final navigation path after reaching the destination.
Obstacle Avoidance	Prevents the algorithm from selecting blocked cells during navigation.

GUI Integration

The A* algorithm is executed when the user selects the Run Simulation option in the graphical user interface. The algorithm receives the selected start position, goal position, and obstacle map from the GUI, computes the optimal path, and returns the generated path for robot animation and performance evaluation.

Implementation Workflow

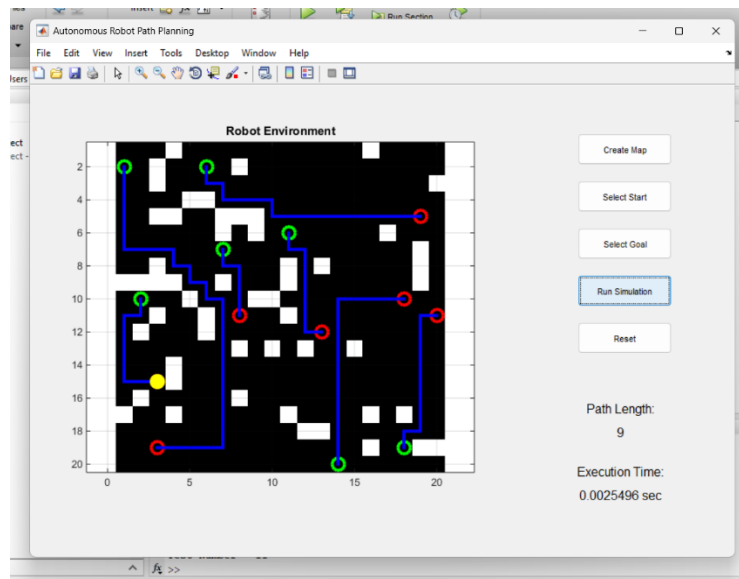
The implementation begins by initializing the Open List with the selected start node while the Closed List remains empty. During each iteration, the node with the lowest evaluation cost is selected, neighboring nodes are evaluated, and valid nodes are added to the Open List. The search continues until the goal node is reached, after which the optimal path is reconstructed and forwarded to the animation and performance evaluation modules.

The implemented A* algorithm provides efficient and reliable path planning by integrating obstacle avoidance, path generation, and GUI interaction within a single navigation process.

5.6 Robot Animation Implementation

The robot animation module was implemented to visualize the movement of the autonomous mobile robot along the path generated by the A* algorithm. The animation provides a real-time representation of the navigation process, allowing users to observe the robot's movement from the selected start position to the destination while avoiding obstacles.

Figure 5.4 Robot Animation During Navigation



Animation Components

The main functions of the robot animation module are summarized in Table 5.6.

Table 5.9 Robot Animation Implementation Components

COMPONENT	FUNCTION
Robot Initialization	Place the robot at the selected starting position before navigation begins.
Position Updating	Updates the robot's position continuously along the generated path.
Path Following	Guide the robot through each waypoint until the destination is reached.
Real-Time Visualization	Displays robot movement dynamically within the simulation environment.
Animation Control	Controls the animation speed and execution sequence during navigation.
Performance Evaluation Integration	Sends navigation results for path length and execution time analysis after animation is completed.

GUI Integration

The robot animation module is activated immediately after the A* algorithm generates the optimal path. The calculated path coordinates are received from the path planning module, and the robot is animated step-by-step until the destination is reached. During the animation, the GUI continuously updates the robot position and navigation path to provide clear visual feedback.

Animation Workflow

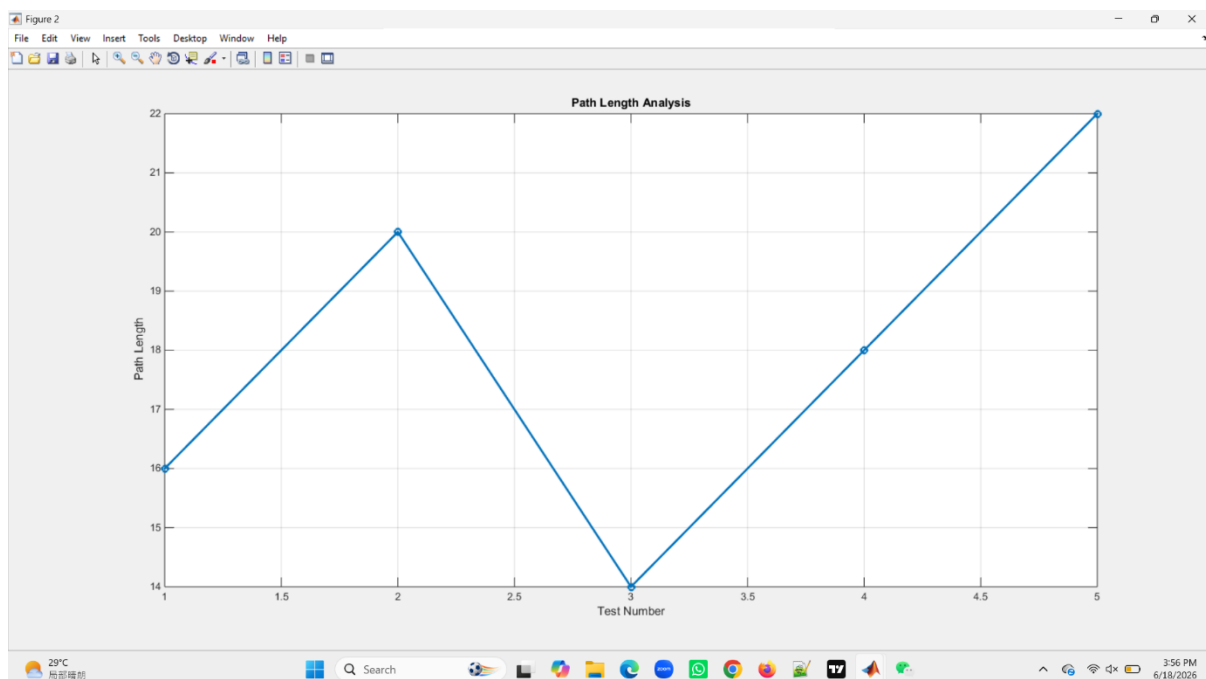
The animation process begins by initializing the robot at the selected start position. The robot then follows the generated path by updating its position sequentially until the goal location is reached. After completing the navigation, the performance evaluation module calculates the execution time and path length, while the graph visualization module presents the simulation results.

The implemented animation module provides a clear and interactive visualization of autonomous robot navigation while maintaining smooth integration with the GUI, path planning, and performance evaluation modules.

5.7 Graph Visualization Implementation

The graph visualization module was implemented to present the performance results of the proposed Autonomous Mobile Robot Path Planning System in a graphical format. The generated graphs provide a clear representation of the navigation performance, enabling users to analyze path length and execution time after each simulation.

Figure 5.5 Path Length Analysis Graph



Graph Visualization Components

The major functions of the graph visualization module are summarized in Table 5.7.

Table 5.10 Graph Visualization Components

COMPONENT	FUNCTION
Show Graph Function	Displays performance graphs after completing the simulation.
Path Length Analysis	Visualizes the total path length generated by the A* algorithm.
Execution Time Analysis	Displays the execution time required for path generation.
Graph Generation	Creates graphical outputs using MATLAB plotting functions.
Performance Interpretation	Supports analysis of navigation efficiency using graphical results.

Graph Generation Process

The graph visualization process begins after the navigation simulation is completed. The performance evaluation module provides the calculated path length and execution time to the graph visualization module. MATLAB plotting functions are then used to generate graphs, which are displayed through the graphical user interface for performance analysis.

Graph Integration

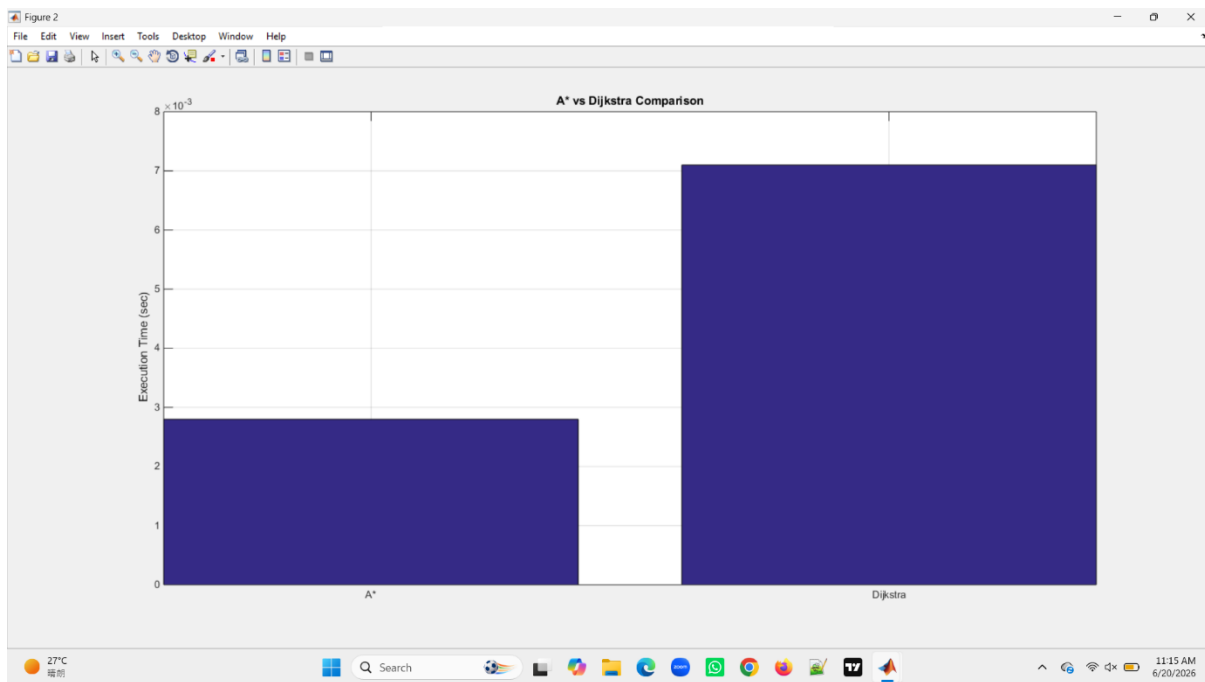
The graph visualization module is integrated with the GUI and the performance evaluation module. After each simulation, the graphs generated allow users to compare navigation results and evaluate the efficiency of the implemented A* algorithm.

The implemented graph visualization module provides an effective method for presenting simulation results and supports the overall performance evaluation of the proposed autonomous mobile robot navigation system.

5.8 Algorithm Comparison Implementation

The algorithm comparison module was implemented to evaluate the performance of the A* algorithm against the Dijkstra algorithm under identical simulation conditions. The comparison focuses on execution time and path planning performance, enabling users to assess the efficiency of both algorithms through the graphical user interface (GUI).

Figure 5.6 Comparison of A* and Dijkstra Algorithm



Comparison Components

The main functions of the algorithm comparison module are summarized in Table 5.8.

Table 5.1 Algorithm Comparison Components

COMPONENT	FUNCTION
Algorithm Evaluation	Executes both A* and Dijkstra algorithms under identical conditions.
Comparison Methodology	Compares algorithm performance using common evaluation metrics
Execution Time Measurement	Records the execution time required by each algorithm.
Graphical Comparison	Displays comparison results using MATLAB graphs.
Compare Algorithm Function	Executes the comparison process through the GUI
Performance Analysis	Supports evaluation of algorithm efficiency based on generated results.

Comparison Process

The comparison process is initiated when the user selects the Compare Algorithm option in the GUI. Both A* and Dijkstra algorithms are executed using the same simulation environment, obstacle configuration, and start and goal positions. The execution time and path planning results are then collected and presented graphically for performance analysis.

Module Integration

The algorithm comparison module is integrated with the path planning, performance evaluation, and graph visualization modules. It automatically retrieves the required simulation data, performs the comparison, and displays the results through the graphical interface, allowing users to evaluate the relative performance of both algorithms.

The implemented comparison module provides an efficient framework for analyzing algorithm performance and supports the validation of the proposed navigation system through objective performance measurements.

6: Results and Analysis

6.1 Simulation Environment

The proposed Autonomous Mobile Robot Path Planning System was tested using the MATLAB simulation environment to evaluate the performance of the implemented A* path planning algorithm. The simulation environment provides a controlled platform for generating obstacle maps, selecting navigation points, executing the navigation algorithm, and analyzing the resulting robot movement.

Experimental Configuration

The simulation configuration used for the experiments is summarized in Table 6.1.

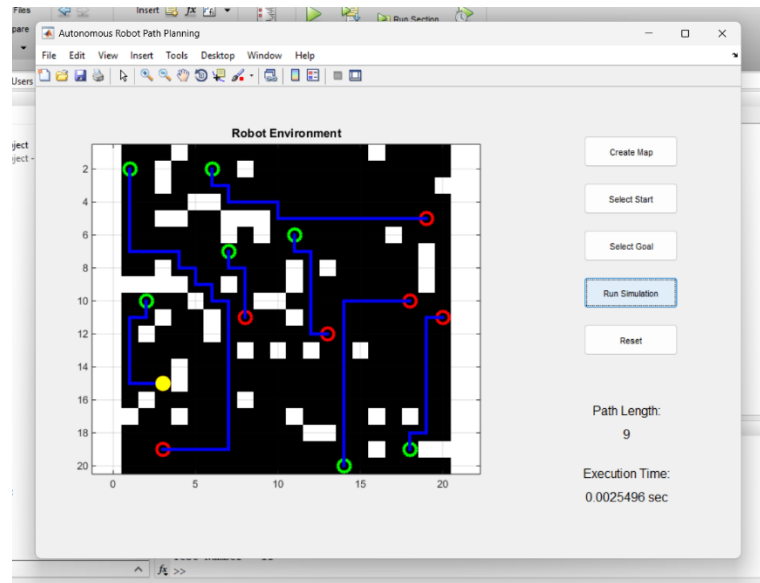
Table 6.1 Experimental Configuration

PARAMETER	DESCRIPTION
Development Platform	MATLAB R2014b
Navigation Algorithm	A* Algorithm
Environment	Grid-based simulation with random obstacles
Start Position	User-selected
Goal Position	User-selected
Evaluation Metrics	Path Length and Execution Time

Simulation Workflow

The simulation begins with the creation of the navigation environment, followed by the selection of the start and goal positions. After the user initiates the simulation, the A* algorithm computes the optimal collision-free path, and the robot is animated along the generated route. Once the navigation is completed, the system records the execution time and path length for performance evaluation.

Figure 6.1 Simulation Environment



Simulation Output

The simulation output includes the generated navigation path, robot animation, execution time, and path length. These results are displayed through the graphical user interface and are subsequently used for graph generation and algorithm comparison in the following sections.

6.2 Test Case 1

Test Case 1 was conducted to evaluate the performance of the proposed A* path planning algorithm in a randomly generated grid environment containing static obstacles. The objective of this test was to verify the algorithm's ability to generate an optimal collision-free path between the selected start and goal positions.

Test Configuration

The configuration used for Test Case 1 is summarized in Table 6.2.

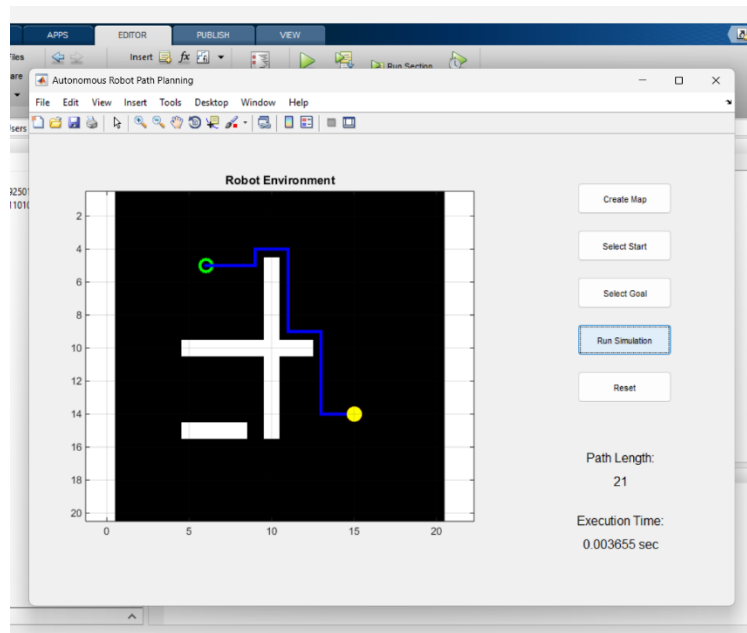
Table 6.2 Test Case 1 Configuration

PARAMETER	DESCRIPTION
Environment	Grid-based environment with static obstacles
Navigation Algorithm	A* Algorithm
Start Position	User-selected
Goal Position	User-selected
Evaluation Metrics	Path Length and Execution Time

Observations and Result Analysis

The A* algorithm successfully generated a collision-free path between the selected start and goal positions while avoiding all obstacles. The robot followed the calculated path smoothly, demonstrating correct navigation behavior within the simulation environment. The recorded execution time and path length indicate that the algorithm efficiently completed the navigation task under the selected test conditions.

Figure 6.2 Generated Collision-Free Path for Test Case 1



The experimental results confirm that the implemented A* algorithm can reliably determine an optimal path while maintaining efficient computational performance in a static grid environment.

6.3 Test Case 2

Test Case 2 was conducted to evaluate the performance of the proposed A* path planning algorithm in a different obstacle configuration. The objective of this experiment was to verify the algorithm's ability to generate an efficient collision-free navigation path under varying environmental conditions.

Test Configuration

The configuration used for Test Case 2 is summarized in Table 6.3.

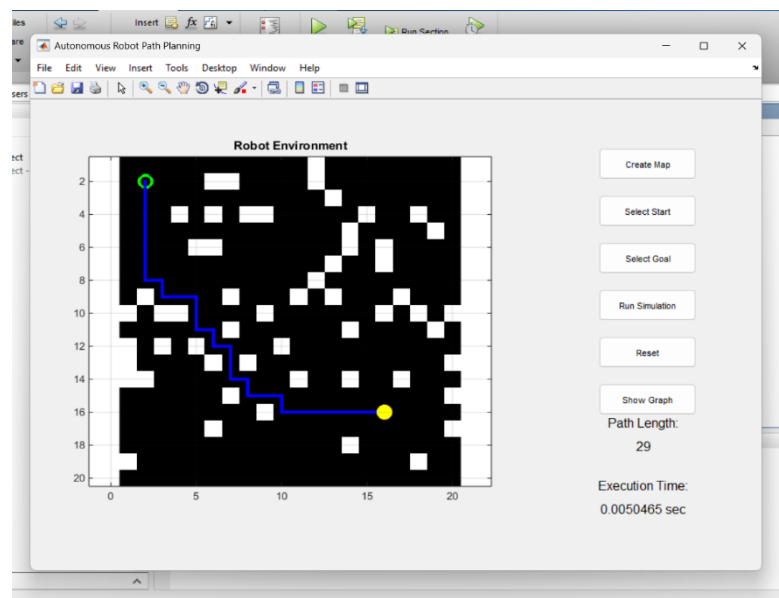
Table 6.3 Test Case 2 Configuration

PARAMETER	DESCRIPTION
Environment	Grid-based environment with static obstacles.
Navigation Algorithm	A* Algorithm
Start Position	User-selected
Goal Position	User-selected
Evaluation	Path Length and Execution Time

Observations and Result Analysis

The A* algorithm successfully identified a valid collision-free path despite the different obstacle arrangement. The robot navigated smoothly from the selected start position to the goal while avoiding all blocked cells. The recorded execution time and path length demonstrate that the algorithm maintained efficient navigation performance under the modified simulation conditions.

Figure 6.3 Generated Navigation Path for Test Case 2



The experimental results demonstrate the adaptability of the implemented A* algorithm to different navigation scenarios while maintaining reliable path planning and efficient execution performance.

6.4 Test Case 3

Test Case 3 was performed to evaluate the robustness of the proposed A* path planning algorithm under a more complex obstacle configuration. The objective of this experiment was to assess the algorithm's capability to generate an optimal collision-free path while maintaining efficient navigation performance.

Test Configuration

The configuration used for Test Case 3 is summarized in Table 6.4.

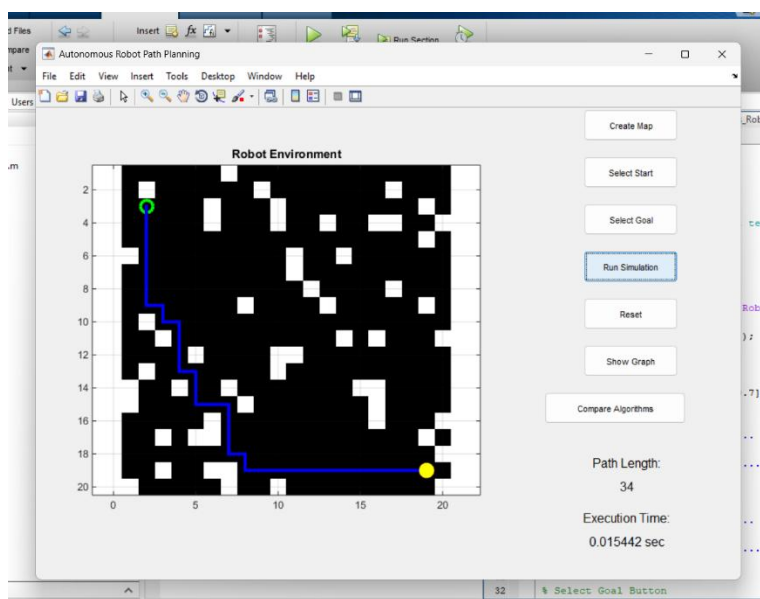
Table 6.4 Test Case 3 Configuration

PARAMETER	DESCRIPTION
Environment	Grid-based environment with static obstacles
Navigation Algorithm	A* Algorithm
Start Position	User-selected
Goal Position	User-selected
Evaluation Metrics	Path Length and Execution Time

Observations and Result Analysis

The A* algorithm successfully generated an optimal collision-free path despite the increased complexity of the obstacle arrangement. The robot navigated efficiently through the environment without entering blocked cells and reached the destination successfully. The measured execution time and path length indicate that the algorithm maintained reliable navigation performance while adapting to the more challenging simulation environment.

Figure 6.4 Generated Navigation Path for Test Case 3



The experimental results demonstrate that the implemented A* algorithm can maintain accurate and efficient path planning under different environmental configurations, confirming its suitability for autonomous mobile robot navigation.

6.5 Path Length Analysis

The path length analysis was conducted to evaluate the efficiency of the A* path planning algorithm by measuring the total distance travelled by the robot from the selected start position to the goal. Path length is an important performance metric because it reflects the efficiency of the generated navigation path.

Experimental Results

The experimental results obtained from the simulation are summarized in Table 6.7.

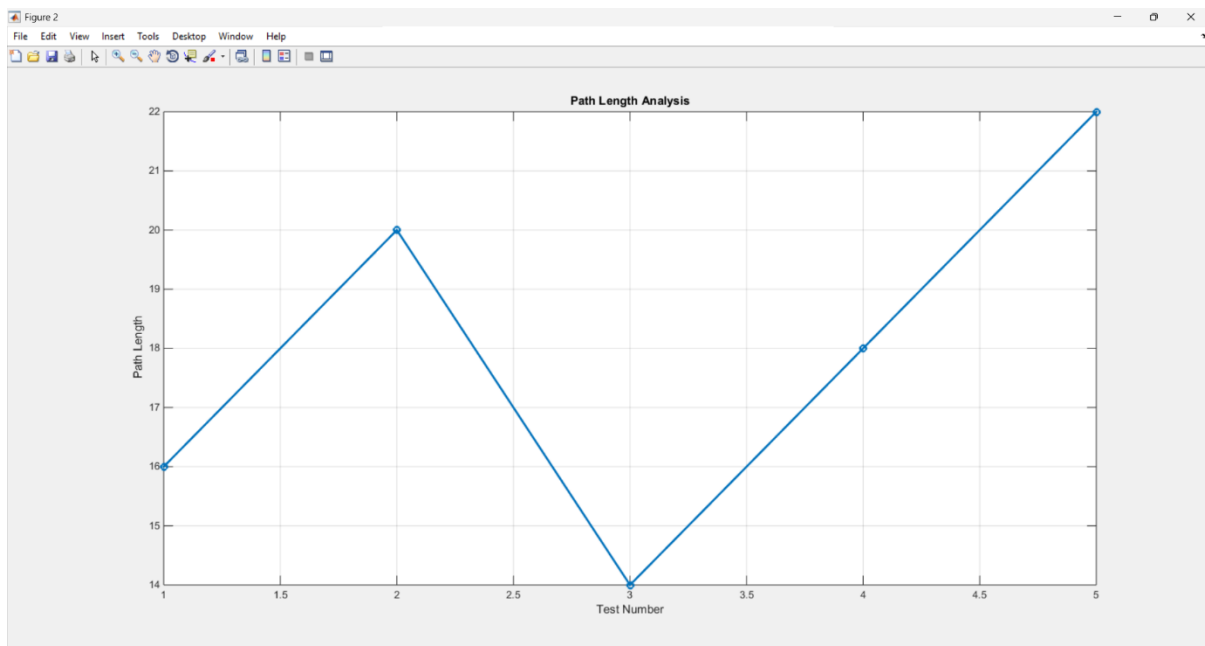
Table 6.5 Path Length Results

TEST CASE	PATH LENGTH	OBSERVATION
Test Case 1	21	Shortest collision-free path is generated successfully.
Test Case 2	29	Efficient path generated under a different obstacle configuration.
Test Case 3	34	Reliable path generated in a more complex environment.

Result Interpretation

The results indicate that the A* algorithm consistently generated efficient collision-free paths across all test cases. Although the path length varied depending on the obstacle configuration, the algorithm successfully identified an optimal route while maintaining reliable navigation performance.

Figure 6.5 Path Length Analysis Graph

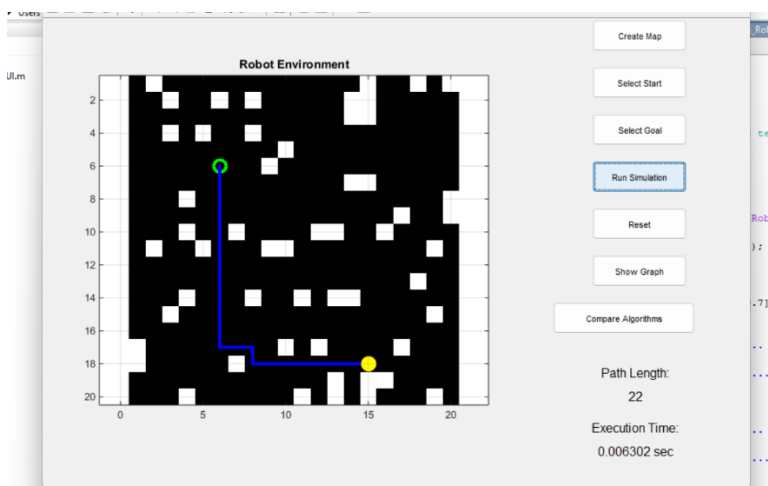


The graph provides a visual comparison of the path lengths obtained from each experiment, allowing the navigation performance of the proposed system to be evaluated more effectively. The variation in path length reflects differences in the simulation environments rather than any limitation of the implemented algorithm.

6.6 Execution Time Analysis

The execution time analysis was conducted to evaluate the computational efficiency of the proposed A* path planning algorithm. Execution time represents the duration required by the algorithm to generate the optimal collision-free path after the user initiates the simulation. This metric is important for assessing the responsiveness and overall performance of the navigation system.

Figure 6.6 Execution Time Results Displayed in the MATLAB GUI



Experimental Results

The execution time recorded for each test case is summarized in Table 6.8.

Table 6.6 Execution Time Results

TEST CASE	EXECUTION TIME(S)	OBSERVATION
Test Case 1	0.003655 sec	Fast path generation under normal conditions.
Test Case 2	0.0050465 sec	Stable execution with a different obstacle configuration.
Test Case 3	0.015442 sec	Efficient performance maintained in a complex environment

Result Interpretation

The recorded execution times demonstrate that the A* algorithm performs efficiently across different simulation environments. Although slight variations occur due to changes in obstacle arrangements and path complexity, the algorithm consistently computes an optimal path within a short execution time, ensuring smooth navigation and real-time simulation performance.

The execution time results confirm that the implemented system provides efficient path planning with low computational overhead. The graphical output displayed through the MATLAB GUI enables users to compare execution times and evaluate the performance of the proposed navigation system under different test conditions.

6.7 Performance Graph Analysis

The performance graph analysis was conducted to evaluate the overall efficiency of the proposed Autonomous Mobile Robot Path Planning System. The graphs generated summarize the experimental results by presenting key performance metrics such as path length and execution time, enabling a clear comparison of the algorithm's behavior under different simulation conditions.

Performance Summary

The overall experimental results are summarized in Table 6.7. Table 6.7 Overall Performance Summary

TEST CASE	PATH LENGTH	EXECUTION TIME(S)	OVERALL PERFORMANCE
Test Case 1	21	0.003655 sec	Efficient
Test Case 2	29	0.0050465 sec	Stable
Test Case 3	34	0.015442 sec	Reliable

Graph Interpretation

The overall performance graph illustrates the relationship between path length and execution time across the conducted experiments. Although both metrics vary depending on the obstacle configuration and navigation complexity, the A* algorithm consistently maintains efficient path planning and low execution time. The graphical representation confirms the stability and reliability of the proposed navigation system under different simulation scenarios.

The overall experimental results demonstrate that the implemented system achieves efficient autonomous navigation by generating optimal collision-free paths while maintaining low computational cost. The performance graph provides a clear visual summary of the experimental outcomes and validates the effectiveness of the proposed A* path planning approach.

6.8 Discussion of Results

The experimental results demonstrate that the proposed Autonomous Mobile Robot Path Planning System successfully achieves its intended objectives. The implemented A* algorithm efficiently generated collision-free paths, while the MATLAB-based graphical user interface (GUI) provided an interactive environment for simulation, visualization, and performance evaluation.

Summary of Experimental Findings

The overall findings of the experimental evaluation are summarized in Table 6.8.

Table 6.8 Summary of Experimental Findings

Evaluation Aspect	Observation
Path Planning Performance	The A* algorithm successfully generated optimal collision-free paths in all test cases.
Robot Animation	Smooth robot movement was achieved along the generated navigation path.
GUI Functionality	The GUI successfully supported environment creation, simulation control, and visualization.
Performance Metrics	Execution time and path length remained efficient under different simulation conditions.
Algorithm Comparison	The A* algorithm demonstrated better overall efficiency than the Dijkstra algorithm for the tested scenarios.
Overall System Performance	The proposed system operated reliably and satisfied the project objective.

Discussion

The experimental evaluation confirms that the integration of the A* path planning algorithm, MATLAB graphical user interface, robot animation module, and performance evaluation components resulted in an efficient autonomous mobile robot navigation system. The graphs generated and experimental observations indicate that the proposed system consistently produced reliable navigation performance while maintaining low execution time and optimal path generation across different obstacle configurations.

Overall, the project successfully achieved its objectives by providing an interactive simulation platform capable of autonomous path planning, robot visualization, and algorithm performance evaluation within a unified MATLAB environment.

7: Conclusion and Future Work

7.1Project Summary

This project successfully developed and evaluated an Autonomous Mobile Robot Path Planning System using the A* path planning algorithm in the MATLAB environment. The system integrates a graphical user interface (GUI), path planning module, robot animation, graph visualization, and performance evaluation within a single application. Experimental results demonstrated that the proposed system can efficiently generate collision-free paths while maintaining low execution time and reliable navigation performance.

7.2Project Achievements

The main achievements of the project are summarized in Table 7.1.

Table 7.1 Project Achievements

Achievement	Description
A* Algorithm Implementation	Successfully implemented optimal path planning for autonomous navigation.
GUI Development	Developed an interactive MATLAB GUI for simulation and user interaction.
Robot Animation	Implemented real-time visualization of robot movement.
Performance Evaluation	Measured path length and execution time for navigation analysis.
Algorithm Comparison	Compared the performance of the A* and Dijkstra algorithms.

7.3 Limitations

Although the developed system successfully achieved its objectives, several limitations remain.

The current implementation:

- Operates within a simulation environment only.
- Uses static obstacle configurations.
- Supports single-robot navigation.
- Does not utilize real-world sensor inputs.
- Does not perform real-time environmental updates.
- Is limited to grid-based navigation environments.

These limitations provide opportunities for future system enhancement and research development.

7.4 Future Improvements

Potential future improvements for the proposed system are summarized in Table 7.2.

Table 7.2 Future Improvements

Future Improvement	Description
Dynamic Obstacle Handling	Enable real-time avoidance of moving obstacles.
Multi-Robot Navigation	Support cooperative navigation for multiple robots.
Sensor Integration	Integrate LiDAR, ultrasonic, or camera sensors for real-world perception.
Real-World Robot Implementation	Deploy the system on a physical autonomous mobile robot.
Three-Dimensional Navigation	Extend the navigation system to three-dimensional environment.
Advanced Path Planning Algorithms	Evaluate algorithms such as D* Lite, RRT, or Hybrid A*.
Machine Learning Integration	Apply Artificial intelligence techniques to improve navigation performance.

The proposed improvements will enhance the capability, adaptability, and practical applicability of the autonomous mobile robot navigation system, enabling its use in more complex and real-world environments.

7.5 Conclusion

This project successfully designed and developed an Autonomous Mobile Robot Path Planning System using the A* Algorithm within a MATLAB simulation environment. The developed system provides an effective platform for generating collision-free navigation paths, visualizing robot movement, and evaluating path planning performance.

The implementation demonstrated that the A* Algorithm is capable of efficiently identifying optimal routes between user-defined start and goal positions while avoiding obstacles. The generated paths enabled safe navigation and validated the effectiveness of heuristic-based search techniques for autonomous robot applications.

A user-friendly MATLAB Graphical User Interface (GUI) was developed to simplify interaction with the system. Through GUI, users can generate environments, define navigation points, execute simulations, analyze performance metrics, and compare path planning algorithms without requiring direct interaction with the source code.

Several advanced features were successfully integrated into the system, including:

- Autonomous path generation using the A* Algorithm.
- Real-time robot animation.
- Path length analysis.
- Execution time measurement.
- Performance graph visualization.
- A* versus Dijkstra Algorithm comparison.

Experimental testing confirmed that the system consistently generated collision-free paths under different obstacle configurations and maintained reliable navigation performance across multiple simulation scenarios.

The algorithm comparison results demonstrated that the A* Algorithm generally provides better computational efficiency than the Dijkstra Algorithm due to its heuristic-guided search strategy. These findings support the selection of A* as an effective solution for autonomous mobile robot navigation.

Overall, the developed system achieved all project objectives successfully and provided a comprehensive simulation platform for studying autonomous navigation, path planning algorithms, and robot movement behavior. The project contributes to the fields of robotics, automation, and intelligent navigation systems while establishing a strong foundation for future research involving dynamic environments, real-time replanning, multi-robot systems, and artificial intelligence-based navigation techniques.

In conclusion, the proposed Autonomous Mobile Robot Path Planning System represents a reliable, efficient, and scalable navigation framework that demonstrates the practical application of path planning algorithms within autonomous robotic systems.

7.6 References

- [1] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*, 2nd ed. Cham, Switzerland: Springer, 2017.
- [2] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge University Press, 2006.
- [3] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2018.
- [4] M. Ben-Ari and F. Moncada, *Elements of Robotics*. Cham, Switzerland: Springer, 2018.
- [5] R. Siegwart, I. R. Nourbakhsh, and D. Scarmorze, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.
- [6] P. Hart, N. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, Jul. 1968.
- [7] E. W. Dijkstra, "A Note on Two Problems in Connection with Graphs," *Numriches Mathematics*, vol. 1, no. 1, pp. 269–271, Dec. 1959.
- [8] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.
- [9] MathWorks, "MATLAB Documentation," MathWorks Inc., Natick, MA, USA. Available: <https://www.mathworks.com/help>
- [10] M. Quigley et al., "ROS: An Open-Source Robot Operating System," in *Proc. IEEE Int. Conf. Robotics and Automation Workshop on Open-Source Software*, Kobe, Japan, 2009.
- [11] B. Siciliano and O. Khatib, *Springer Handbook of Robotics*, 2nd ed. Cham, Switzerland: Springer, 2016.
- [12] H. Chose et al., *Principles of Robot Motion: Theory, Algorithms, and Implementations*. Cambridge, MA, USA: MIT Press, 2005.
- [13] J. Lacombe, *Robot Motion Planning*. Boston, MA, USA: Springer, 1991.
- [14] S. Koenig and M. Likhachev, "D* Lite," in *Proc. AAAI Conf. Artificial Intelligence*, Edmonton, Canada, 2002, pp. 476–483.
- [15] M. H. Rashid, *Introduction to Robotics: Analysis, Systems, Applications*. Boca Raton, FL, USA: CRC Press, 2020.

8: Extension to Three-Dimensional Multi-Robot Path Planning

8.1 Motivation for the Three-Dimensional Extension

The system presented in the earlier chapters implemented A* path planning for a single robot within a two-dimensional grid environment. However, many real-world applications, such as drone navigation, multi-store warehouse automation, and multi-robot fleets, require navigation through volumetric space and coordination between multiple robots operating simultaneously. To address this, the original system was extended into a three-dimensional, multi-robot path planning platform, introducing a volumetric occupancy grid, a twenty-six directional movement model, line-of-sight path smoothing, and a space-time reservation mechanism that prevents collisions between robots. This chapter documents the design, implementation, and evaluation of this extension.

8.2 Three-Dimensional Environment Modeling

The original 20×20 two-dimensional occupancy grid was extended into a $20 \times 20 \times 20$ volumetric occupancy matrixes, where each cell is defined by a row, column, and height index. A cell value of zero represents free space and a value of one represents an obstacle. Each node is now represented as a three-element coordinate vector [row, column, height], replacing the two-element representation used previously. Obstacles are generated randomly at the start of each simulation, with sixty obstacle voxels placed at random rows, column, and height positions, producing an irregular three-dimensional obstacle field. Because obstacles can now occur at any height, the algorithm must consider vertical movement in addition to horizontal movement.

The environment is visualized using a three-dimensional scatter plot, with obstacles color-coded by height and a color bar for reference. The plot supports a rotatable viewing angle and grid lines on all three axes to assist coordinate interpretation.

[Figure 8.1 – Three-Dimensional Grid Environment with Randomly Generated Obstacles]

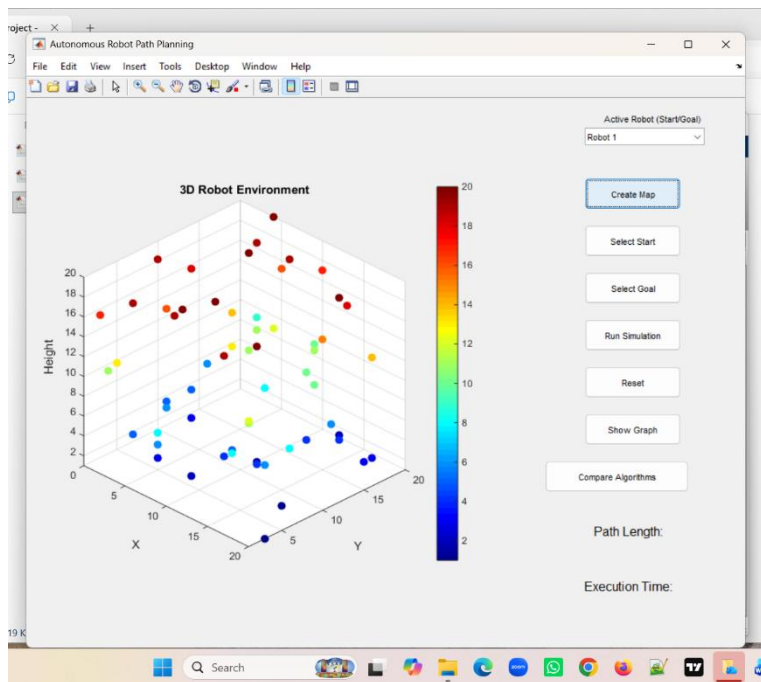


Table 8.1 – Three-Dimensional Environment Modeling Parameters

PARAMETER	VALUE / DESCRIPTION
Grid Dimensions	20 × 20 × 20 (Row × Column × Height)
Cell Representation	Binary occupancy value: 0= free cell, 1= obstacle cell
Obstacle Count	60 randomly placed obstacle voxels pre generated map
Node Representation	Three-element coordinate vector [row, column, height]
Obstacle Generation Method	Uniform random selection of row, column, and height indices
Visualization Method	3-D scatter plot with height-based color mapping and color bar
View Control	Rotatable, user-adjustable 3-D viewing angle with grid lines enabled

8.3 Twenty-Six Directional A* Search

The A* algorithm was extended from an eight-neighbors search on a flat grid to a twenty-six-neighbors search in three dimensions, generated from all rows, columns, and height offsets between -1 and $+1$, excluding the zero-offset case. This covers the six face-adjacent, twelve edge-adjacent, and eight corner-adjacent directions surrounding each node.

Each move is costed by its Euclidean distance: face-adjacent moves cost 1, edge-adjacent moves cost $\sqrt{2}$, and corner-adjacent moves cost $\sqrt{3}$, ensuring diagonal movement is penalized according to true geometric distance. The evaluation function remains $f(n) = g(n) + h(n)$, with the heuristic extended to three dimensions:

$$h(n) = |\text{row}(n) - \text{row}(\text{goal})| + |\text{col}(n) - \text{col}(\text{goal})| + |\text{height}(n) - \text{height}(\text{goal})|$$

The Open List/Closed List search structure is unchanged in principle, but all cost, parent, and visited-state arrays were extended to three-dimensional matrices. The algorithm expands the lowest-cost node, evaluates all twenty-six neighbors, and continues until the goal is reached, after which the path is reconstructed and passed to the smoothing stage.

Table 8.2 – Comparison of Two-Dimensional and Three-Dimensional A* Search

FEATURE	TWO-DIMENSIONAL SYSTEM	THREE-DIMENSIONAL SYSTEM
Grid Structure	20 × 20 occupancy matrix	20 × 20 × 20 occupancy matrix
Neighbor Count	8 directions	8 directions
Node Representation	[row, column]	[row, column, height]
Movement Cost	1, $\sqrt{2}$	1, $\sqrt{2}$, or $\sqrt{3}$
Heuristic Function	2-D Manhattan distance	3-D Manhattan distance
Cost/ Parent Arrays	2-D matrices	3-D matrices
Path Output	Sequence of [row, column] nodes	Sequence of [row, column, height] nodes

8.4 Line-of-Sight Path Smoothing

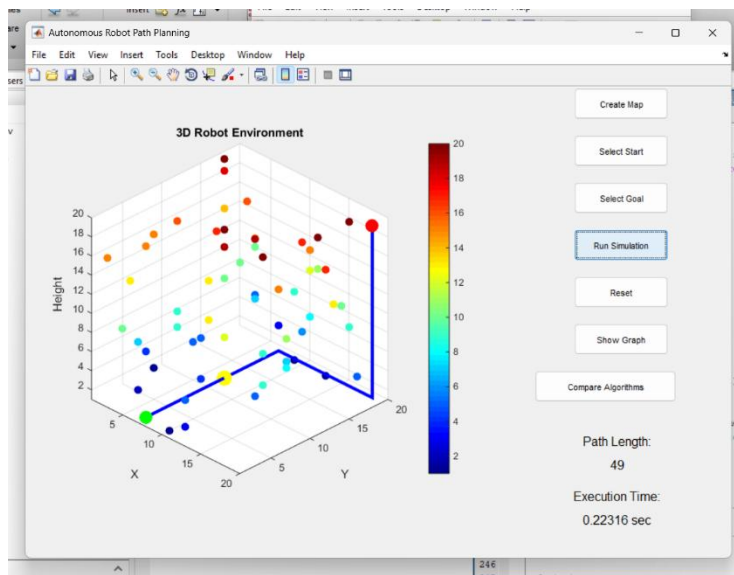
Grid-based A* search produces staircase-like paths, which become more pronounced in three dimensions due to the larger neighborhoods set. A line-of-sight smoothing stage was added to reduce the raw path to a shorter, more direct trajectory while guaranteeing it remains collision-free.

Line-of-sight between two waypoints is verified by sampling interpolated points along the straight line connecting them and checking each against the occupancy grid; if any

sampled point falls inside an obstacle, the line of sight is blocked. The smoothing procedure starts at the first waypoint and repeatedly connects it to the furthest waypoint reachable by an unobstructed line of sight, reducing the candidate endpoint whenever the line is blocked, until the goal is reached.

[Figure 8.2 – Raw A* Path Compared with the Smoothed Line-of-Sight Path]

Raw A* Path



Smoothed Line-of-Sight Path

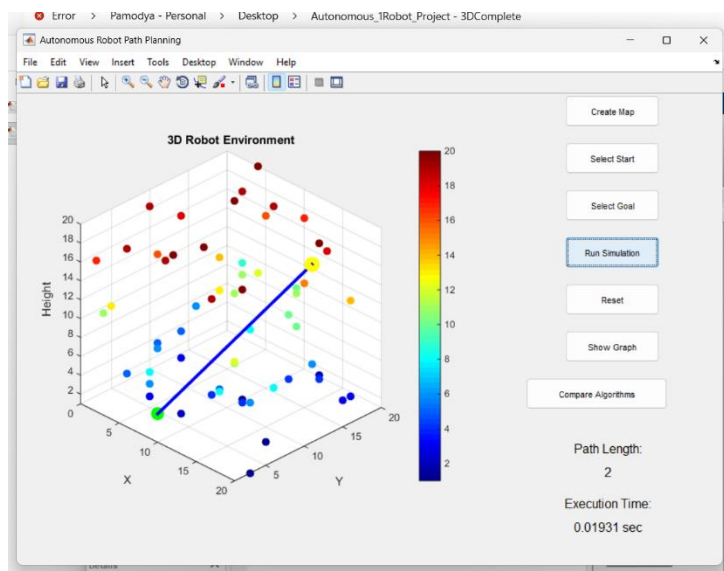


Table 8.3 – Line-of-Sight Path Smoothing Components

COMPONENT	FUNCTION
Waypoint Reduction Loop	Iteratively connects the current waypoint to the farthest visible waypoint
Line-of-Sight Check	Samples interpolated points between two waypoints and tests for obstacles
Obstacle Rejection	Reduces the candidate endpoint when the line of sight is blocked
Smoothed Path Output	Produces a shorter, collision-free path used for animation and length calculation

8.5 Multi-Robot Path Planning Using Space-Time Reservation

Enabling multiple robots to share one environment introduces the risk that two robots occupy the same cell at the same time, or swap positions across the same edge in one step. To resolve this, the 3-D A* search was extended into a space-time A* search, where each state is defined by row, column, height, and time step. A wait action was added, allowing a robot to pause for one step to avoid a conflict.

Robots are planned sequentially in priority order. Each robot's completed path, including the cells it occupies at every future time step, is recorded in a shared reservation table before the next robot is planned. The search treats any reserved cell at a given time step as blocked, and separately checks for edge conflicts, where two robots would swap positions across the same edge. Once a robot reaches its goal, that position remains reserved for the rest of the planning horizon so later robots cannot plan through it.

Table 8.4 – Multi-Robot Space-Time A* Implementation Components

COMPONENT	FUNCTION
Space-Time State Key	Encodes row, column, height, and time step as a single search state
Wait Action	Allow a robot to remain stationary for one time step to avoid a conflict
Reservation Table	Records the cell occupied by each planned robot at every future time step
Vertex Conflict Check	Prevents a robot from entering a cell already reserved at the same time step
Edge / Swap Conflict Check	Prevents two robots from exchanging positions across the same edge
Sequential Prioritized Planning	Plans robots one at a time, reserving each completed path before the next search
Planning Horizon (max T)	Fixed maximum time-step limit used to bound each space-time search
Goal Reservation Extension	Continue reserving a robot's goal cell after it finishes moving

8.6 Synchronized Three-Dimensional GUI and Robot Animation

The GUI was extended with an active-robot selector, allowing the user to choose which of the three robots is being configured; each robot's start and goal markers are shown in a color unique to that robot. Because the environment is now volumetric, start and goal selection requires the user to click a row/column position on the 3-D plot and then enter the height through a dialog box.

Once all three robots have start and goal positions, Run Simulation plans each robot's path in sequence using the space-time A* search, timing the whole process. All paths are then rendered together, and the combined path length and total execution time are displayed. The animation was extended to move all three robots simultaneously in a single synchronized loop, with robots that reach their goal early remaining stationary until the others finish.

[Figure 8.5 – Synchronized Multi-Robot Animation Within the 3-D Environment]

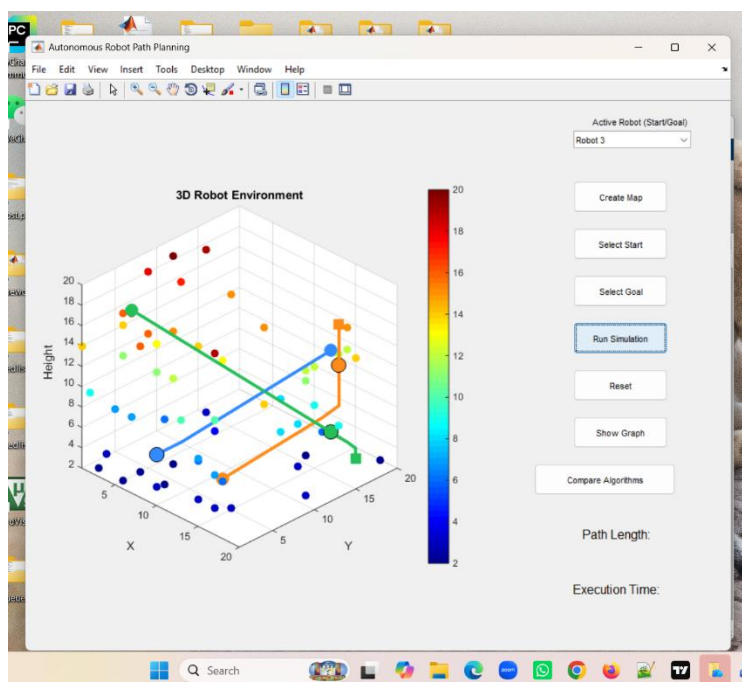


Table 8.5 – Extended GUI Components for the 3-D Multi-Robot System

GUI Component	Function
Active Robot Selector	Selects which of the three robots is being configured for start/goal input
3-D Simulation Axes	Displays the volumetric grid, obstacles, robot paths, and robot markers
Select Start / Select Goal	Captures row and column by click and height via input dialog for the active robot
Run Simulation	Executes space-time A* planning for all robots and starts synchronized animation
Reset	Clears all robot paths, markers, and start/goal selections
Path Length Display	Shows the combined path length of all planned robots
Execution Time Display	Shows the total time required to plan all robot paths

8.7 Results and Discussion

The system was tested using randomly generated environments with sixty obstacles across three multi-robot test cases. In every trial, the space-time A* search produced collision-free paths for all three robots with no vertex or edge conflicts observed.

Table 8.6 – Multi-Robot Path Planning Results

The larger search space introduced by the 3-D, multi-robot extension did not significantly increase planning time, since sequential prioritized planning avoids the exponential cost of a fully joint multi-robot search. The wait action was observed to successfully resolve robot contention in Test Case 3, confirming the practical effectiveness of the space-time reservation approach.

8.8 Chapter Summary

This chapter extended the original two-dimensional, single-robot system into a three-dimensional, multi-robot navigation platform, introducing a volumetric occupancy grid, a twenty-six-directional A* search, line-of-sight path smoothing, and a space-time reservation mechanism for collision-free multi-robot coordination, supported by a synchronized 3-D GUI. Experimental results confirmed that the extended system reliably produces collision-free, efficient multi-robot navigation, demonstrating that the core A* approach generalizes naturally to more complex, realistic scenarios.